



中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

学 校

参赛队号

	1.
队员姓名	2.
	3.

中国研究生创新实践系列大赛
“华为杯”第二十三届中国研究生
数学建模竞赛

题目：通用神经网络处理器的多核切图与调度优化

摘要：

面向具有多条计算与搬运管道的通用神经网络处理器，多核切图必须同时权衡计算并行、跨子图通信、同步等待与共享存储竞争。本文将算子依赖图划分为子图，联合确定核心映射和核上顺序，以赛题评估程序给出的总完成时间为主目标，以额外数据搬运量为同时时间下的次目标。三个问题分别对应跨子图均经 DDR 的独立 Task、允许同核复用的跨核同步 Task，以及新增共享 FIFO 只读 L2 的场景；我们在共同决策空间上逐层加入运行机制。

对问题一，建立保持依赖无环的有向图划分与多核列表调度模型，从完整分量、局部切分和共享输入关系构造结构不同的初解，再对关键区域做有界联合调整。100 张图的逐图平均加速比在 1 至 5 核下分别为 1.000、1.835、2.546、3.161、3.666；五核总完成时间为 73170501 周期。对问题二，模型显式约束跨核 COPY 完成后的 500 周期释放，并按同核驻留与四条管道的事件执行评价。在同一批图上，五核逐图平均加速比为 4.160，总完成时间为 65769876 周期；相对本工程上一轮完整方案，400 个多核配置中 13 组改善、387 组持平。

对问题三，进一步跟踪共享 L2 的 FIFO 队列，把命中搬运与 DDR 搬运分配到各自共享带宽池，并区分固定调度方案的硬件效应与重新调度收益。五核逐图平均加速比达到 4.240，总完成时间为 64562282 周期；但 500 组逻辑额外搬运合计增加 4418288 字节，且仍有 4 组比无 L2 对照更慢。三个场景均完成 100 图、1 至 5 核的逐组官方评价与结果复核。结果说明，缓存命中或搬运减少不能单独代表执行时间收益，方案优劣应回到完整依赖和资源事件下的最终完成时间判断。

关键词：有向无环图划分；多处理器调度；离散事件仿真；共享缓存

1 问题重述

1.1 背景与任务

通用神经网络处理器以多个 AI 核执行计算图。每核具有矩阵计算、向量计算与数据搬运等独立执行通道，但核内存储有限，多个核心共享核外 DDR 带宽。原始计算图为带张量依赖的有向无环图；切分粒度决定可同时运行的工作量，也改变输入重复读取、跨子图数据搬运和缓存溢出。题目要求输出每个非 COPY_IN/COPY_OUT 算子的子图编号，以及各核心上的子图顺序，再由给定评估程序确定总执行周期和额外搬运量^[1]。

三个问题的决策形式相近，运行语义逐步改变。问题一的场景 A 规定一个子图就是一个 Task，子图边界上的数据总经 DDR 中转；即使两个 Task 在同一核上连续执行，也不保留私有缓存。问题二的场景 B 把同核各子图纳入一个 Task，允许核内驻留，但跨核数据需等待同步释放。问题三在场景 B 上增加全核共享的只读 FIFO L2，读请求可由 DDR 或 L2 服务。因而，问题二和三不能直接沿用场景 A 的通信成本公式，也不能把问题三的缓存命中率当作独立可选变量。

1.2 所需回答与指标

对每张图和每个核数 $k = 1, \dots, 5$ ，本文构造合法的切图、核心分配与核上顺序。首要评价量为官方仿真得到的完成时间 $T_{i,k}$ (cycles)，同时报告逻辑额外数据搬运量 $D_{i,k}$ (bytes)。设 $T_{i,1}^0$ 是题目规定的整图单核核内启发式结果，则前两问的逐图加速比与平均加速比定义为

$$S_{i,k} = \frac{T_{i,1}^0}{T_{i,k}}, \quad \bar{S}_k = \frac{1}{100} \sum_{i=1}^{100} S_{i,k}. \quad (1)$$

这是一百个比值的算术平均，不能换成总单核周期除以总多核周期。问题三另比较相同图、相同核数下的无 L2 与有 L2 周期，单核缓存收益由真实仿真决定，不人为设为 1。

1.3 研究范围

全部实验使用题目给定的 100 张计算图和固定硬件参数。本文不改变图的算子、张量或官方核内启发式；搜索仅改变题目允许的划分、映射和子图顺序。求解器运行秒数与被评估方案的执行周期是不同量，前者反映求解效率，后者是模型的主要目标。各问的逐例结果列于附录，所有整体结论均以同一百图为统计总体。

2 问题分析与总体路线

2.1 关键权衡

合并强依赖算子可以减少边界 COPY、重复读取和任务释放等待，但过度聚合会压缩多核并行空间，并可能让核内启发式在 L1/UB 容量压力下插入更多换入换出。过度切分则使共享 DDR 竞争、同步及启动间隔增加。因此，通信字节数、任务数或各核算子数都不是总完成时间的充分代理。若 Cube 与 Vector 工作量不均，按算子个数平衡也会让其中一条 Pipe 长期空闲。论文以合法方案的官方 Makespan 为最终准则，将成本估计只用于有限候选的生成和排序。

三问共同采用“结构候选—依赖调度—官方评价—有界改进”的框架。先识别计算依赖和共享输入，构造完整连通分量、主导分量有界拆分等不同粒度的合法初解；再按后续关键工作量和最早可行完成时间安排核心与空隙；最后用少量局部划分、迁移和顺序调整寻找更短的总时间。场景 B 的候选必须重新按同核驻留、跨核 COPY 释放和容量规则评价；共享 L2 的候选还需经过 FIFO 事件仿真。算法保留已知可行方案，在限定评价机会内比较，不进行分组、核心和排列的组合穷举。

2.2 三种运行语义的关系

表 1 概括后两问相对前一问的改变。相同的输出字段不意味着相同的代价函数：尤其在场景 B 中，一个核心内的多个子图仍有先后次序，但其通信可利用驻留，跨核释放以具体数

据搬运事件为准。问题三缓存只改变合资格读取的服务路径，不能绕过算子依赖、同步和四条 Pipe 的占用。

表 1 三问运行语义与增量机制

内容	问题一：场景 A	问题二：场景 B	问题三：B 加共享 L2
Task 与驻留	一子图一 Task；边界不驻留	同核子图同属一 Task；可驻留	保持 B 的 Task 与私有存储
跨核数据	边界经共享 DDR；按 A 释放	COPY 完成后另有同步释放	同 B，不因命中取消同步
读取服务	DDR	DDR	COPY_IN 可命中共享 FIFO L2
主要决策	划分、映射、核上序	同左，重估驻留和生命周期	同左，重估缓存事件与双带宽池

3 模型假设与数据准备

3.1 假设及适用依据

假设输入图与官方固定配置正确给出算子、张量、周期、依赖和硬件参数；算子不能被拆成更细的可调度原子，张量大小与算子周期在一次评价内确定。忽略 CPU 主控本身未由官方评价器计入的时间，不把网络传输、温度或动态频率引入模型。每个方案需满足题目规定的覆盖、无环、核内顺序、L1/UB 容量及资源占用约束。若容量压力触发溢出，新增 COPY 由原始核内启发式确定，不能由外层算法任意删除。

使用的 100 张图由原附件逐字节无损提取，没有删除异常图、归一化周期或调整张量大小。导入时对全部图执行官方结构校验，并保存源文件与每份模型输入的 SHA-256；所有问题读取同一处理目录。由此，三问结果的差异来自场景语义及合法方案，而非改写数据。前期挑选小图和反例用于调试与同预算对照；最终汇总只以完整百图为准，不把开发样本当独立盲测集。

3.2 固定硬件量

L1 容量为 524 288 bytes，UB 容量为 131 072 bytes，共享 DDR 总带宽为 60 bytes/cycle。场景 A 的跨核任务等待为 1000 cycles，同核连续 Task 间隔为 100 cycles；场景 B 的跨核 COPY 释放延迟为 500 cycles。问题三的共享只读 L2 容量为 1 048 576 bytes，带宽为 250 bytes/cycle。以上均是主实验的固定配置，灵敏度扫描仅用于解释机制，不替换正式结果。

4 符号说明

5 问题一：无核间同步机制下的切图与调度

5.1 场景 A 的组合优化模型

令 V_c 为所有非 COPY 算子。映射 π 把它们恰好分配给 q 个非空子图， m 把每个子图分配给一个核心， σ_c 给出核心 c 上各子图的顺序。把跨子图的数据前驱构成商图，再加入每核顺序边，所得扩展商图必须无环；否则即使原图无环，核心顺序仍可能与跨核等待构成死锁。合法方案集合记为 $\mathcal{P}_A(k)$ 。对于每份方案，官方内层启发式先产生实际 COPY 和核内 Pipe 序，再进行共享 DDR 事件仿真。我们以此可计算但不便线性化的函数定义目标：

$$\min_{p \in \mathcal{P}_A(k)} \text{lex} (T_i^A(p), D_i(p)), \quad T_i^A(p) = \max_{u \in \mathcal{T}(p)} f_u. \quad (2)$$

这里的字典序表示优先缩短最后一个 Task 的完成时间；仅在周期相同时比较额外搬运。它是算法的选优规则，不把较少搬运误说成必然较短的周期。

表 2 主要符号

符号	含义
$G = (V, E), V_c$	原计算 DAG 及其中的非 COPY 算子集合
$\pi : V_c \rightarrow \{1, \dots, q\}$	操作至子图的划分; q 为子图数
$m : \{1, \dots, q\} \rightarrow \{1, \dots, k\}$	子图至核心的映射; σ_c 为核心 c 上的合法顺序
$p = (\pi, m, \sigma)$	一份完整调度方案; $\mathcal{P}_A, \mathcal{P}_B$ 为对应场景的合法方案集
$T_i^A(p), T_i^B(p), T_i^{L2}(p)$	第 i 图在三种语义下的官方完成周期
$D_i(p)$	相对原始图 COPY 总量的逻辑额外搬运字节数
$Q(t), C, b_D, b_L$	时刻 t 的共享 FIFO 队列、L2 容量、DDR 与 L2 带宽
h_e, s_e, a_e, f_e	COPY 事件 e 的命中标志、字节数、发射和完成时刻

Task v 的开始须满足所有数据前驱释放和同核前一 Task 的间隔。以 F_u 表示前驱完成时刻, $P_c(v)$ 表示核心 c 上的前一 Task, 可写为

$$R_v \geq \max_{u \in \text{pred}(v)} (F_u + 1000 \mathbf{1}\{m_u \neq m_v\}), \quad R_v \geq F_{P_{m_v}(v)} + 100. \quad (3)$$

同核依赖不额外施加跨核的 1000 周期, 但跨 Task 张量仍须由 DDR 中转。公式 (3) 仅描述释放约束, 不能单独算出时长; Cube、Vector、搬运 Pipe 的先后关系、L1/UB 溢出以及同时在途的 DDR 传输会改变 F_u 。实际评价必须采用题目给出的事件过程。

为了识别一份输入被多个子图重复读取的情形, 我们把共享张量与多个消费算子的关系视为有向超边, 保留原图方向和商图无环性。多层次无环划分研究说明, 此类表示比仅统计两两割边更适合描述共享输入^[2]。我们借鉴这一表示和粗化思想, 但没有照搬其平衡目标或宣称实现了文献的完整算法; 本题的最终目标仍是式 (2)。

5.2 结构感知初解与有界搜索

首先构造整图保底方案与依赖贪心方案。对没有计算依赖的弱连通分量, 尝试保持完整并按估计的最长 Pipe 工作量分核; 这样能释放天然并行度, 且减少不必要的分量内切边。若一个主导分量明显压过其余分量, 则在满足无环性的拓扑窗口中有限拆分, 以免完整保留造成单核长尾。共享输入复用、局部核内时长和容量压力同时参与候选排序; 局部估计只负责提出方案, 不能替代官方全局仿真。

将候选子图调度到核心时, 先按后续关键工作量排序, 再为每个子图寻找最早合法插入空隙和预计完成时间较小的核心。该思想与最早完成时间列表调度相近^[3], 但本题必须附加式 (3)、共享 DDR 和真实容量条件。随后在固定的评价机会内, 尝试少量关键区域的边界调整、核心迁移与顺序修复; 每个被提出的完整方案均检查操作覆盖和扩展商图无环, 并由原始评价语义选优。

当前采用方案对每个多核配置限制为 12 次评价机会、确定性种子 0。基础粒度搜索保留机会, 结构新候选须通过事件成本准入; 失败的候选不能因预测好就替代实测较优方案。当前研究版本启用了局部成本、关键路径、一般映射空隙插入、通信排序、划分保护、共享输入及有限区域修复。实验中测试过而未被采用的事件池整体重排序、扩大的自适应组合等, 不计入本轮主结果。固定预算使评价调用数随图数线性增长; 候选构造与图规模相关, 但不枚举指数数量的分组、映射和任务排列。

5.3 百图结果与解释

表 3 和图 1 给出固定单核分母下的百图结果。五核的逐图平均加速比为 3.665903, 总执行周期为 73 170 501; 后二者分别衡量“每图等权速度”和“这批图的总工作量”, 不可相互替代。二至五核的总周期随核数降低, 但加速不是线性的: 更多核心既释放并行, 也可能增加 DDR 读取和跨核等待。

表 3 问题一当前方案的百图汇总

核心数	平均加速比	总周期 (cycles)	总额外搬运 (bytes)
1	1.000000	312 479 373	1 241 686 120
2	1.834917	158 746 394	1 972 107 414
3	2.546238	109 880 862	1 848 970 300
4	3.161009	86 330 529	1 764 926 118
5	3.665903	73 170 501	1 767 728 284

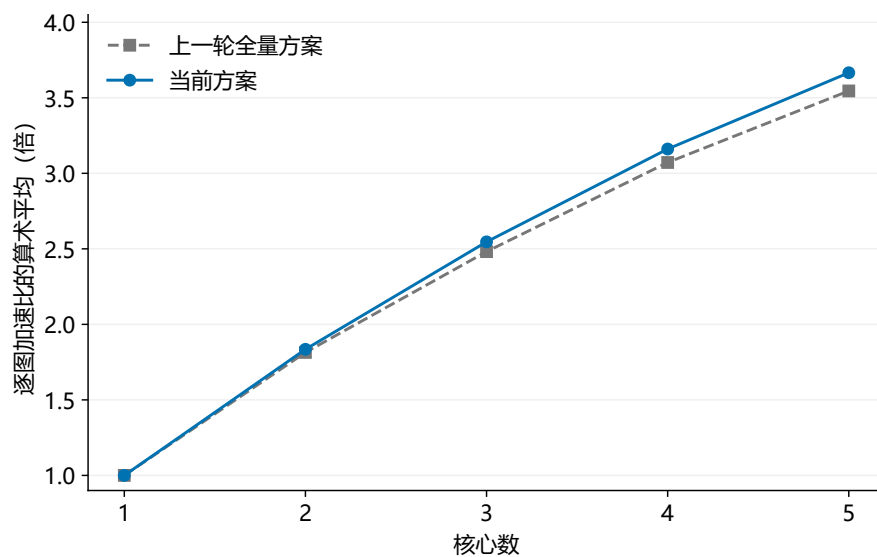


图 1 问题一的百图逐图加速比算术平均

与本工程上一轮相同百图方案相比，二至五核总周期分别下降 0.9958%、1.6807%、2.7519%、3.6678%；400 个多核配置中 68 胜、330 平、2 负。负例是第 6 图的三核和四核，不能把各阶段收益简单相加，或说每张图都改善。第 59 图在二、四、五核取得本批次最大的逐图加速比，三核最大值来自第 97 图；这些仅为分布端点，不替代百图均值。由于单核分母本身是题目规定的核内启发式而非全局最优，少数多核比值超过核数可以出现，不构成评价错误。

五百份方案均通过官方结构及顺序校验，逐行指标与保存的官方评价相符；代表图还做了冷启动复现。大图采用增量无环检查、状态索引和已固定准备的复用，以减少求解器自身时间；这些工程措施不应被解释成 NPU 任务周期的额外收益。当前热基准搜索最长约 364 秒，但它不含所有配置的冷单核生成和额外最终复核，因此不能断言从零开始每图均在十分钟内结束。完整逐例周期与搬运见附录。

6 问题二：同核驻留与跨核同步下的调度

6.1 场景 B 的模型扩展

问题二仍选择 (π, m, σ) ，但场景 B 不再把每个子图独立清空为一个 Task：映射到同一核心的全部子图组成该核的 Task，核内合法顺序决定缓存复用与张量生命周期。对于同一张量，若生产和消费能够在同核满足依赖，则中间结果可以驻留；跨核消费必须按实际生产核与消费核构造搬运，并在源 COPY_OUT 完成后经过 500 cycles 才能释放目标 COPY_IN。对于跨核数据事件 e ，有

$$a_e^{\text{in}} \geq f_e^{\text{out}} + 500. \quad (4)$$

这里的前驱是具体搬运事件，而非“源子图全部运行完”的粗粒度时刻。对多消费者共享张量，外部输入按消费核心而非每个算子独立读取；跨核中间张量按实际源核—目标核关系生成，避免把每条逻辑边简单等同一次物理 COPY。

合法集合 $\mathcal{P}_B(k)$ 除保留覆盖、子图商图和核上顺序的无环性外，还要求由官方核内展开得到的物理实例满足 L1/UB 在每个时刻的容量约束。四条 Pipe 各自串行，但不同 Pipe 在依赖允许时可重叠；同一 DDR 池服务所有在途外存搬运。张量能否驻留依赖实际产生、消费和溢出时刻，不能只按静态张量大小之和判断。令 $\mathcal{E}_B$ 表示题目的事件模拟，问题二的目标为

$$\min_{p \in \mathcal{P}_B(k)} \text{lex} (T_i^B(p), D_i(p)), \quad T_i^B(p) = \mathcal{E}_B(G_i, p; k). \quad (5)$$

这仍是依赖官方事件过程的组合优化模型，不把启发式估价器当作精确求解的线性目标。与问题一相比，映射会同时改变通信、驻留、释放和并行，所以切图后固定映射、或仅按分组时长排序，均可能错过有效方案。处理器互连与片上存储的耦合也是相关体系结构研究关注的问题^[4]；本题仍以给定的缓存及 COPY 语义为准，不移植其他硬件的通信规则。

6.2 继承初解与有限联合调整

问题二的每份初始方案先以场景 B 官方规则复评，不能沿用场景 A 的周期作为成绩。求解器保留完整分量、共享输入顺序、主导分量切分、链及深度窗口等结构候选，并对场景 B 重新估计同核驻留和跨核传输。候选的核心及顺序安排借鉴依赖任务的最早完成时间框架，但处理重叠 Pipe、共享 DDR 和式 (4) 时，代理只能用于排序，最终比较仍由完整官方评价完成。

在结构初解之后，普通有界邻域把少数相关子图迁移到另一核心，并在合法位置插入；也可在不改变主要结构的前提下修复局部顺序。这样同时改变“任务在哪里执行”和“何时可以开始”，而不把所有 k^q 种映射或各核全排列列举出来。当前采用的规则在每个多核配置中保留已有效的基础候选，回收重复候选占用的评价机会，总场景 B 机会仍为 12 次。对于主导计算分量明显的图，结构准入开放相应候选；未达到结构条件时维持原有基础机会。该统一规则在全部百图上执行，不在看到某图结果后另换一套策略。

初始问题一方案只是合法起点，问题二另有自己的评价与改进。搜索中所有完整候选

先做图结构、核上顺序与内存合法性检查；重复方案计入机会台账，不伪装成额外自由搜索。已有方案作为保底，后续方案仅按实测周期优先、同周期较少逻辑搬运的规则接受。研究期间测试过关键链引导邻域、通信代理改写和生命周期面积排序，但它们并未全部进入本轮采用配置，因此不能分别给这些未采用机制分摊收益。

6.3 百图结果与验证

表 4 显示五核逐图平均加速比为 4.159974，百图总周期为 65 769 876。两至五核在相同固定单核参照下均完成一百张图；图 2 为规定的平均加速比曲线。总额外搬运量没有随核心数严格单调变化，表明较短 Makespan 与较少逻辑 COPY 不总能同时获得。

表 4 问题二当前方案的百图汇总

核心数	平均加速比	总周期 (cycles)	总额外搬运 (bytes)
1	1.000000	312 479 373	1 241 686 120
2	1.981113	153 876 051	1 128 799 934
3	2.815714	103 980 830	1 079 358 470
4	3.550822	79 744 878	1 092 834 382
5	4.159974	65 769 876	1 087 391 086

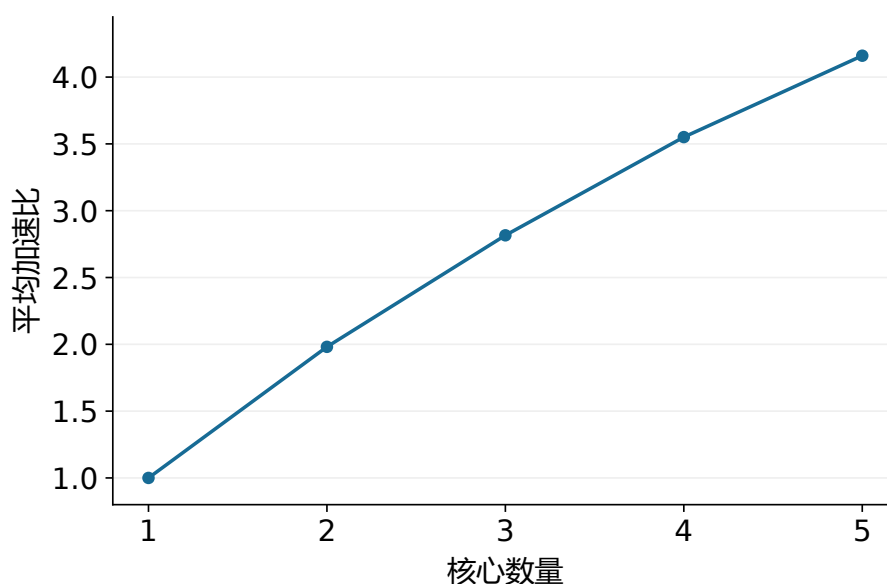


图 2 问题二的百图逐图加速比算术平均

相对本工程上一轮完整百图方案，当前配置在 400 个多核组合中 13 胜、387 平、无退步；这些组合总周期从 403 398 480 降为 403 371 635，降幅约 0.006655%。同时，额外搬运合计由 4 387 116 718 增至 4 388 383 872 bytes，增加约 0.028884%。这一小幅变化来自整套候选选择与机会分配，不能表述为某一局部算子带来的独立效果，也不应掩盖次指标的代价。

四百份多核方案均由场景 B 官方评价器最终回放，物理张量生命周期容量检查通过，非法候选未进入交付结果；一百份单核点使用题目规定的固定整图基准。五百份标准方案与逐例周期、搬运和加速比一同保存。个别图在增加核心数后仍出现周期上升，原因可能是同步及带宽代价抵消新增并行；因此“核心越多每例必然越快”不是模型约束。本问的改进仅对给定百图与固定配置成立，未提供全局最优证书或独立未见图的泛化保证。

7 问题三：共享 FIFO 缓存下的多处理器调度

7.1 缓存状态与双带宽事件模型

问题三继承场景 B 的合法方案集 $\mathcal{P}_B(k)$ 、跨核释放与核内内存约束，新增容量为 C 的共享只读 L2。它不是每核各有一份，也不改变已生成的逻辑 COPY 数量。对时刻 t 按进入先后排序的逻辑张量队列记为 $Q(t)$ ，有

$$\sum_{x \in Q(t)} s_x \leq C, \quad C = 1048576 \text{ bytes.} \quad (6)$$

正字节 COPY_IN 事件 e 在发射时刻 a_e 查询逻辑张量标识。若标识已在 $Q(a_e^-)$ 中则 $h_e = 1$ ，改由 L2 服务；否则 $h_e = 0$ ，仍由 DDR 服务。命中不刷新 FIFO 次序。搬运在完成时按官方处理顺序进行填充：未驻留且大小不超过 C 的张量，从队首淘汰至有足够空间后入队；超容量张量旁路。两个同时发射的冷读不能因为较早一个尚在途中而合并为一次命中。完成时的重新插入、相同时间事件次序等均保持官方语义。

设 $A_D(t)$, $A_L(t)$ 分别为 DDR 和 L2 池的在途操作集合， $b_D = 60$ 、 $b_L = 250$ bytes/cycle。每条搬运按所属池的独占带宽形成初始服务工作量，活跃集合不变的时间段内，各自池内公平分享服务：

$$\dot{w}_e(t) = -\frac{1}{|A_r(t)|}, \quad e \in A_r(t), \quad r \in \{D, L\}. \quad (7)$$

事件到达或完成后须结算剩余工作并重算完成时刻；实际评价还依官方整数周期取整。两池独立，不能把 $60 + 250$ 当作共同带宽，也不能给每个核心各分配一条 250 bytes/cycle 的 L2 通道。L2 命中只改变 COPY 的服务池，不能绕过 (4) 中的跨核同步，不能占用别的 Pipe 插队，亦不能使超出 L1/UB 的物理实例合法。

于是问题三在相同合法方案集上求

$$\min_{p \in \mathcal{P}_B(k)} \text{lex}(T_i^{L2}(p; C, b_L), D_i(p)). \quad (8)$$

字节命中率为

$$H_i = \frac{\sum_{e: \text{COPY_IN}, s_e > 0} h_e s_e}{\sum_{e: \text{COPY_IN}, s_e > 0} s_e}. \quad (9)$$

分母为零时只在统计程序中定义为零并标注“无有效读取”。官方命中率按字节而非按次数统计。 D_i 是插入额外逻辑 COPY 的字节量，不因命中自动扣减；同一方案分别在无 L2 和 L2 下评价，该字段应相同。命中字节可作为实际 DDR 服务减量的辅助诊断，但 H_i 越高未必总周期越短，因为同步、计算、L2 带宽竞争以及关键路径变化仍可能占主导。

7.2 固定方案消融与缓存事件引导搜索

首先将问题二的 500 份方案在无 L2 的场景 B 与启用 L2 的场景下分别官方重放。两次使用相同划分、映射和顺序，因此其差异刻画固定方案上的硬件效应。随后以该 Q2 方案为保底，在真实 L2 时间线中定位反复未命中的张量与相关消费子图，有限尝试合法的核心迁移和空隙插入；普通关键度候选保留为后备。新方案再由完整 L2 官方事件评价，并按周期优先、同周期较少逻辑搬运接受。每个配置最多评估四个新增候选；继承 Q2 种子的既有求解成本与本问新增成本分别记账。

这种拆分避免把“硬件新增 L2”与“因 L2 改变调度”混成一个不可解释的总效应。固定方案比较不需要宣称算法等预算公平；新增邻域是否优于另一种搜索方式则必须在同一候选机会下单独验证。三张开发图的同预算普通邻域对照为二胜六平一负，因此本文只报告最终统一规则的百图成绩，不把该小样本写成普遍优势。开发集中将本问候选上限从四增为十二也未使九个配置的采用周期继续下降，这只是局部预算检查，不是搜索收敛证明。

7.3 百图结果与指标边界

表 5 把无 L2、固定 Q2 方案启用 L2 和重新调度后的 L2 三列并列。前三列加速比均使用同一个无 L2 整图单核基准；后两列则先在每张图、相同核数上计算时间比，再对百图取算术平均。故“五核两个平均加速比之比”不等于“逐图同核时间比的平均”。五核优化 L2 的平均加速比为 4.240375，无 L2 对照为 4.159974；其逐图同核时间比均值为 1.022567。

表 5 问题三的百图成对评价（各比值均逐图计算后平均）

核数	无 L2 加速比	同方案 L2	优化 L2	同核硬件比	同核综合比
1	1.000000	1.008622	1.008622	1.008622	1.008622
2	1.981113	1.984989	1.986721	1.001750	1.002467
3	2.815714	2.828518	2.834213	1.004810	1.006671
4	3.550822	3.586810	3.598194	1.011699	1.014923
5	4.159974	4.227368	4.240375	1.019090	1.022567

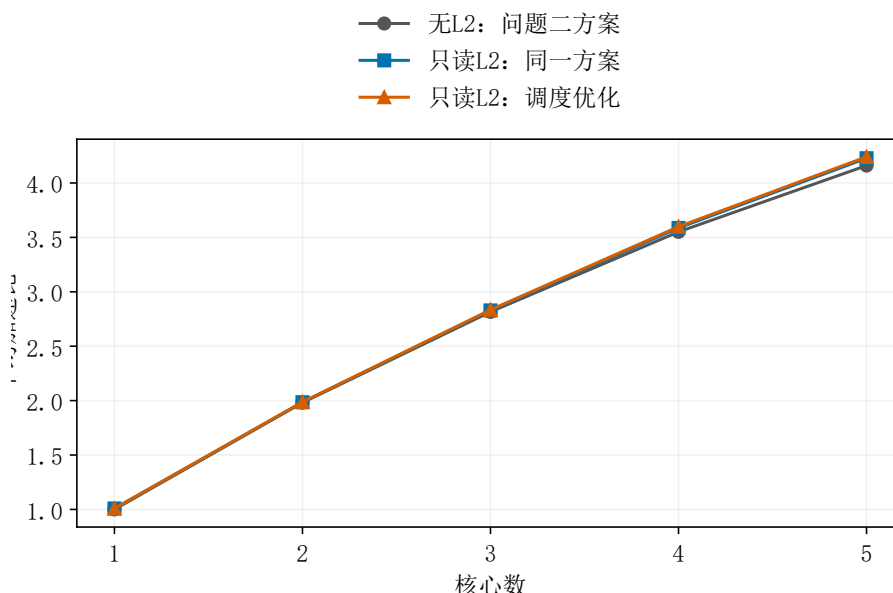


图 3 无 L2、固定方案 L2 与优化方案 L2 的平均加速比

五核百图总周期在无 L2 时为 65 769 876，固定方案启用 L2 后为 64 721 316，进一步调整方案后为 64 562 282。全 500 个配置中，新增搜索使 68 组周期降低，其他组保留不劣的已评价方案；它不保证相对于无 L2 都改善。固定方案启用 L2 有六组退步，最终优化方案仍有四组比无 L2 慢，不能隐藏。全五百组逻辑额外搬运量从无 L2 方案的 5 630 069 992 增至优化 L2 方案的 5 634 488 280 bytes，增加 4 418 288 bytes；这种次指标代价与周期优先的目标一致，但应在方案比较时明确呈现。图 4 给出按同核配对的收益，而不是从总体命中率直接推导周期百分比。

7.4 反例、灵敏度与检验

第 44 图单核从无 L2 的 154 407 降至 107 814 cycles，216 次命中、4 408 416 命中字节全部来自溢出后的重新读取，说明共享 L2 的作用不局限于原始输入复用。以同一方案改变容量时，结果见表 6。这是事后机制诊断，揭示阈值与局部非单调，不是独立确认实验。FIFO 对容量不具备一般的命中集合嵌套性质，相关经典讨论可参见 Belady 等^[5]；本题具体数值只来自自己的官方事件重放。

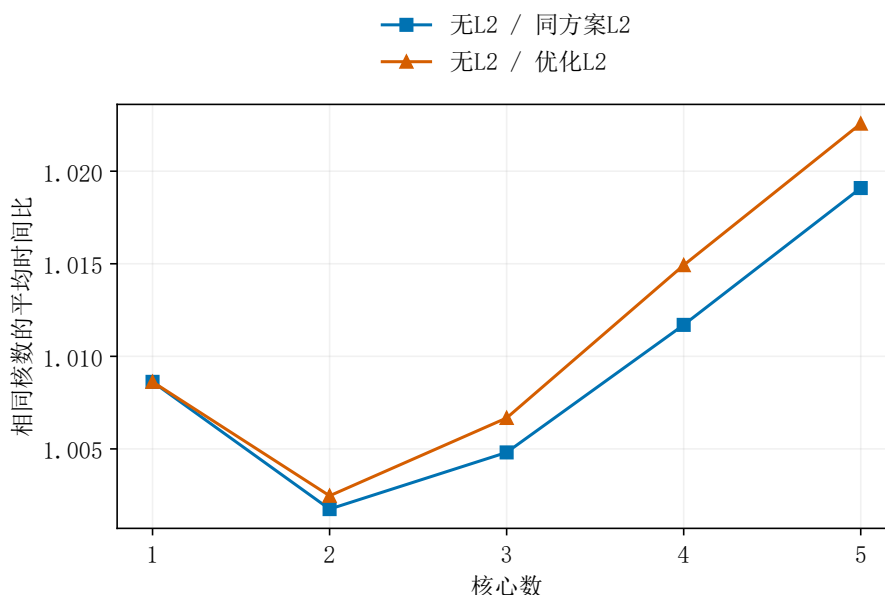


图 4 相同核数下的 L2 硬件效应与调度后综合时间比

表 6 第 44 图单核固定方案的 L2 容量检查

容量 (KiB)	0	64	128	256	512	1024	2048
周期	154407	154272	154407	154407	134726	107814	107814

第 5 图五核固定方案在 L2 带宽分别为 60、125、250、500 bytes/cycle 时，周期分别为 48 200、48 202、48 259、48 253。带宽变化会改变发射与完成次序，进而改变冷读重叠、FIFO 状态和关键路径；不能从“服务速度变快”直接断言固定计划 Makespan 必单调。主实验始终使用题目固定的 1 MiB、250 bytes/cycle。最明显的固定配置退步为第 67 图双核：无 L2 32 040 130，有 L2 32 217 720 cycles。时间线复核将差异定位于搬运关键路径，但该观测分解并非独立因果效应估计。

全部 500 组均完成无 L2/L2 官方成对评价、胜出方案再次回放、FIFO 事件账本和全局物理内存检查；无 L2 周期与问题二冻结指标逐项相同。官方测试集上的回归一致性强于单个演示图，却仍不是未见图的泛化保证。本模型解释固定参数下的结果，未求得所有合法划分上的全局最优，也未把仿真周期当作真实芯片实测时间。

8 模型检验与敏感性分析

8.1 结果口径与可复核性

三个问题均固定相同的 100 张计算图和 1 至 5 核配置，逐组保存算法方案与赛题评估输出。单核加速比的基准采用题目要求的启发式核内调度结果，而非数学最优值；平均加速比先在每张图上求单核与多核时间之比，再对 100 张图取算术平均。该值与单核总周期除以多核总周期不同，后者更突出大图的绝对时间贡献，本文同时报告两者，但只将前者作为赛题指定指标。额外搬运量均沿用官方评估输出的定义，不将缓存命中率或代理估计值混入结果。

每个问题的 500 组方案均经过格式、依赖与官方评估复核。问题一的固定交付证据为 20260924-A-q1-delivery-r02；问题二为 20260924-A-q2-delivery-r07；问题三为 20260924-A-q3-delivery-r03。本文表格和曲线从这些冻结记录生成。对局部优化，保留原方案作为候选，只有官方评价确认改善才替换；这避免由代理排序误差直接造成无条件退步，但不能保证跨场景结果单调，也不能证明全局最优。

8.2 核数与任务粒度

图 1、2 和 3 表明三种场景的平均加速比总体随核数增加。增核同时提高并行能力，也会增加跨子图边、同步释放点或共享资源竞争，因此单图并非必然单调。问题二记录中有 12 个相邻核数配置出现较多核更慢的情况；问题三为 10 个。对这类图，单纯增加核数不是优化动作，应重新联合检查划分、核映射和执行顺序。

子图粒度的作用具有双向性。小子图提供更多可并行任务与调度空隙，但增加跨图数据搬运和任务释放等待；大子图削减边界代价，却可能压缩多核并行度，也可能提高核内活跃数据量并触发换入换出。因此本文生成分量整体、分量内切分及局部边界调整等不同结构的候选，并使用官方完成时间选择，不预设某个算子数量为最优粒度。对大图采用有界候选筛选和局部复用，目的是减少重复核内排程开销；搜索阶段的计时不能代替冷启动、单核基准构建和最终复核在内的端到端计时。

8.3 共享缓存参数与评价目标

问题三对典型图进行了 L2 容量和带宽扫描，而非对 100 图全部硬件参数做全组合搜索。在案例 `case_044` 的单核检查中，缓存命中可消除部分溢出重读，使完成时间从 154407 周期降至 107814 周期；这一大幅收益不应外推至全部图。相反，`case_005` 五核在已检查的带宽值 60、125、250、500 下分别得到 48200、48202、48259、48253 周期，表现为近似持平且略有非单调；`case_067` 两核的 L2 方案为 32217720 周期，高于对应无 L2 方案的 32040130 周期。此类负例说明，缓存降低逻辑搬运与缩短关键路径是两个不同命题，L2 仲裁、传输重叠和任务释放均须进入评价。

问题三固定配置下，所选 L2 方案的 500 组逻辑额外搬运合计比无 L2 对照多 4418288 字节；有 4 组方案时间也高于无 L2。本文因此不以减少搬运量或提高命中率单独宣称性能收益。论文报告的结论只对应给定的硬件配置、FIFO 规则、官方评价程序和当前 100 图集合；若换用真实芯片带宽、缓存替换策略或不同图分布，需重新校准并复核。

9 模型评价与结论

9.1 模型特点

本文以同一计算图表示为基础，按三个场景逐步扩展状态与约束：场景 A 把每个子图视为独立 Task 并令跨图数据经 DDR；场景 B 引入同核复用和跨核同步；场景 C 进一步显式记录共享 L2 的 FIFO 状态与带宽竞争。这样的分层建模保留了问题之间可比较的划分、映射和顺序决策，又避免将某场景不存在的通信或缓存收益误用于另一场景。

求解采用结构化初解、关键路径优先调度与有界局部修复。连通分量、共享输入、双计算管道负载和缓存压力共同决定候选的结构；关键链等待与资源空隙决定局部调度；官方评估程序负责最终合法性和完成时间裁决。通过保留原方案、冻结预算及记录逐组结果，算法可区分预测收益与实际收益，适合在 100 图、1 至 5 核的固定配置上复现。

9.2 局限与改进方向

该方法是启发式算法，没有已证明的最优下界。局部成本模型虽考虑资源阶段和事件传播，仍可能错误排序候选；对于共享 DDR 的并发传输、同步链重构和 L2 替换，局部预测尤其敏感。有限官方评价预算保护了运行时间，却也限制了复杂图的探索范围。问题一的搜索阶段最长约 364 秒，但大图的冷启动单核基准与最终复核未包含在该数值中，故目前不能宣称所有用例的端到端求解都满足 5 至 10 分钟目标。

后续改进应以冻结候选池为前提比较代理排序与官方排序，再针对误差最大的资源事件校准；对关键输出附近采用保持无环的联合边界、映射、顺序调整；并为不同瓶颈结构分配搜索预算。任何新增机制须与原算法同预算、同图、同核数对照，且逐组核验官方完成时间和额外搬运量。若目标转向其他 NPU 或内存层级，需替换相应执行事件和资源容量，而不能直接沿用本文的数值参数。

9.3 结论

在指定 100 图与固定硬件配置下，本文完成三种场景的 500 组方案求解与官方评估。场景 A 的五核平均加速比为 3.665903，场景 B 为 4.159974，场景 C 所选 L2 方案为 4.240375。它们对应不同的通信、同步与缓存规则，不应把三者差值解释成单一算法模块的纯收益。三问共同表明：有效的多核方案取决于切图通信、关键路径等待、核内计算与共享存储竞争的联合平衡；最终判断依据是官方模拟的总完成时间，额外搬运量作为同时间下的次级目标。

参考文献

- [1] 中国研究生数学建模竞赛组委会. 通用神经网络处理器下的多核调度问题 [Z]. 2026.
- [2] Popp M, Schlag S, Schulz C, Seemaier D. Multilevel Acyclic Hypergraph Partitioning[EB/OL]. arXiv:2002.02962, 2020.
- [3] Topcuoglu H, Hariri S, Wu M Y. Performance-effective and low-complexity task scheduling for heterogeneous computing[J]. IEEE Transactions on Parallel and Distributed Systems, 2002, 13(3): 260–274. DOI: 10.1109/71.993206.
- [4] Liu Y, Xue Y, Cheng Y, et al. Scaling Deep Learning Computation over the Inter-core Connected Intelligence Processor with T10[C]//Proceedings of the 30th Symposium on Operating Systems Principles. 2024. DOI: 10.1145/3694715.3695955.
- [5] Belady L A, Nelson R A, Shedler G S. An anomaly in space-time characteristics of certain programs running in a paging machine[J]. Communications of the ACM, 1969, 12(6): 349–353. DOI: 10.1145/363011.363155.

A 核心程序与复现入口

本附录直接从仓库引用求解源文件，避免论文中的程序片段与实际运行代码分叉。完整环境、输入和官方评价代码随交付仓库提供；下面列出三问的核心实现。百图批量运行、校验与结果打包分别由 `q1_delivery_r02.py`、`q2_delivery_r07.py` 和 `q3_submit.py` 及对应交接单约定，命令应在仓库根目录和该版本依赖环境中执行。以下列表为求解器源码，不把开发阶段的实验脚本视为所交付算法。

A.1 问题一求解器

程序 1 问题一求解器

```
1  """问题一初版：贪心融合、多层次有向超图粗化/细化、ALNS。
2
3  估计模型只负责生成候选，最终选择始终使用未修改的官方评估器。
4  """
5  from collections import defaultdict
6  import copy
7  import heapq
8  import json
9  import math
10 import random
11 import time
12
13
14 def topological(pred, succ, priority=None):
15     degree = {u: len(p) for u, p in pred.items()}
16     ready = [(0 if priority is None else -priority[u], u) for u in pred if
17              not degree[u]]
18     heapq.heapify(ready)
19     order = []
20     while ready:
21         _, u = heapq.heappop(ready)
22         order.append(u)
23         for v in sorted(succ[u]):
24             degree[v] -= 1
25             if degree[v] == 0:
26                 heapq.heappush(ready, (0 if priority is None else -
27                                     priority[v], v))
28         if len(order) != len(pred):
29             raise ValueError('子图收缩或执行顺序产生环')
30     return order
31
32 class Graph:
33     def __init__(self, graph, settings, waits):
34         from evaluation_validation import validate_graph
35         from stub_multicore_cut_and_schedule import _build_op_adjacency,
36         _contract_excluded_copy_nodes
37         validate_graph(graph)
38         self.raw = graph
39         self.ops = {o['id']: o for o in graph['ops'] if o['op'] not in {'
40 COPY_IN', 'COPY_OUT'}}
41         _, full = _build_op_adjacency(graph)
```

```

39     self.pred, self.succ = _contract_excluded_copy_nodes(sorted(self.
ops), full)
40     self.order = topological(self.pred, self.succ)
41     self.pos = {u: i for i, u in enumerate(self.order)}
42     self.bandwidth = settings['bandwidth']
43     self.capacity = settings['capacity']
44     self.cross = waits['task_cross_core_wait_cycles']
45     self.same = waits['task_same_core_wait_cycles']
46     all_ops = {o['id']: o for o in graph['ops']}
47     producers, consumers = defaultdict(set), defaultdict(set)
48     for e in graph['edges']:
49         a, b = e['source'], e['target']
50         if a in all_ops and b not in all_ops:
51             producers[b].add(a)
52         elif a not in all_ops and b in all_ops:
53             consumers[a].add(b)
54     self.tensors = []
55     self.incident = defaultdict(list)
56     for t in graph['tensors']:
57         ps = producers[t['id']] & self.ops.keys()
58         cs = consumers[t['id']] & self.ops.keys()
59         if not ps and not cs:
60             continue
61         final = any(all_ops[u]['op'] == 'COPY_OUT' for u in consumers[
t['id']])
62         item = (t['size'], 'UB' if t['pos'] == 'DDR' else t['pos'], ps
, cs, final)
63         index = len(self.tensors)
64         self.tensors.append(item)
65         for u in ps | cs:
66             self.incident[u].append(index)
67     self.rank = {}
68     for u in reversed(self.order):
69         self.rank[u] = self.ops[u]['cycles'] + max((self.rank[v] for v
in self.succ[u]), default=0)
70
71     def view(self, mapping):
72         if set(mapping) != set(self.ops):
73             raise ValueError('算子覆盖不完整')
74         groups = defaultdict(set)
75         for u, s in mapping.items():
76             groups[s].add(u)
77         pred = {s: set() for s in groups}
78         succ = {s: set() for s in groups}
79         for u in self.order:
80             for v in self.succ[u]:
81                 a, b = mapping[u], mapping[v]
82                 if a != b:
83                     succ[a].add(b)
84                     pred[b].add(a)
85         order = topological(pred, succ)
86         return groups, pred, succ, order

```

```

87
88 def costs(self, mapping):
89     """四 Pipe 负载、按 tensor/子图去重的边界流量和活跃集压力代理。"""
90     groups, pred, succ, order = self.view(mapping)
91     loads = {s: defaultdict(float) for s in groups}
92     events = {s: defaultdict(lambda: defaultdict(int)) for s in groups
93 }
94     traffic = 0
95     for u, s in mapping.items():
96         loads[s][self.ops[u]['pipe']] += self.ops[u]['cycles']
97     for size, pos, ps, cs, final in self.tensors:
98         pg, cg = {mapping[u] for u in ps}, {mapping[u] for u in cs}
99         for s in sorted(pg | cg):
100             local_p, local_c = ps & groups[s], cs & groups[s]
101             incoming = bool(local_c) and not local_p
102             outgoing = bool(local_p) and (final or not cs or bool(cs -
103 groups[s]))
104             if incoming:
105                 loads[s]['PIPE_MTE2'] += math.ceil(size / self.
106 bandwidth)
107                 traffic += size
108             if outgoing:
109                 loads[s]['PIPE_MTE3'] += math.ceil(size / self.
110 bandwidth)
111                 traffic += size
112             uses = [self.pos[u] for u in local_p | local_c]
113             # 原图拓扑顺序上的存活区间，仅作压力代理；实际溢出由官方处理。
114             events[s][min(uses)][pos] += size
115             events[s][max(uses) + 1][pos] -= size
116         duration = {}
117         for s in groups:
118             live, peak = defaultdict(int), defaultdict(int)
119             for i in sorted(events[s]):
120                 for pos, delta in events[s][i].items():
121                     live[pos] += delta
122                     peak[pos] = max(peak[pos], live[pos])
123             pressure = sum(max(0, peak[p] - cap) for p, cap in self.
124 capacity.items())
125             duration[s] = max(loads[s].values(), default=0) + 2 * pressure
126             / self.bandwidth
127         return duration, traffic, (groups, pred, succ, order)
128
129 def schedule(self, mapping, cores):
130     duration, traffic, (_, pred, succ, order) = self.costs(mapping)
131     rank = {}
132     for s in reversed(order):
133         rank[s] = duration[s] + max((self.same + rank[v] for v in succ
134 [s]), default=0)
135     end, assigned, free = {}, {}, [None] * cores
136     schedules = [[] for _ in range(cores)]
137     for s in topological(pred, succ, rank):
138         options = []

```

```

132         for c in range(cores):
133             start = 0 if free[c] is None else free[c] + self.same
134             for p in pred[s]:
135                 start = max(start, end[p] + (self.cross if assigned[p]
136 != c else 0))
137             options.append((start + duration[s], c))
138             finish, c = min(options)
139             end[s], assigned[s], free[c] = finish, c, finish
140             schedules[c].append(s)
141             plan = {'node_to_subgraph': {str(u): mapping[u] for u in sorted(
mapping)},
142                     'core_schedules': schedules}
143             proxy = max(max(end.values(), default=0), traffic / self.bandwidth
144 )
145         return plan, proxy
146
147 def greedy_partition(g, cores, block_size=64):
148     """按全局就绪集构造拓扑连续块，优先融合具有数据复用的节点。"""
149     degree = {u: len(g.pred[u]) for u in g.ops}
150     ready = {u for u in g.ops if degree[u] == 0}
151     heap = [(-g.rank[u], u) for u in ready]
152     heapq.heapify(heap)
153     mapping, block = {}, 0
154     target = max(1, min(block_size, math.ceil(len(g.ops) / max(1, 4 *
cores))))
155     while ready:
156         touched, frontier = set(), set()
157         for _ in range(target):
158             if not ready:
159                 break
160             candidates = frontier & ready
161             def score(u):
162                 reuse = sum(g.tensors[i][0] for i in g.incident[u] if i in
163 touched)
164                 return (reuse / g.bandwidth + g.same, g.rank[u], -u)
165             if candidates:
166                 u = max(candidates, key=score)
167             else:
168                 while heap and heap[0][1] not in ready:
169                     heapq.heappop(heap)
170                 _, u = heapq.heappop(heap)
171             ready.remove(u)
172             touched.update(g.incident[u])
173             frontier.discard(u)
174             for i in g.incident[u]:
175                 _, _, ps, cs, _ = g.tensors[i]
176                 frontier.update(v for v in ps | cs if v not in mapping)
177             mapping[u] = block
178             for v in sorted(g.succ[u]):
179                 degree[v] -= 1
180                 if degree[v] == 0:

```

```

179         ready.add(v)
180         heapq.heappush(heap, (-g.rank[v], v))
181     block += 1
182     return mapping
183
184
185 def multilevel(g, initial, cores, levels=3):
186     """有向超图粗化，保存层次，再以近似调度代价选择性展开细化。"""
187     mapping = dict(initial)
188     hierarchy = []
189     for _ in range(levels):
190         groups, pred, succ, _ = g.view(mapping)
191         affinity = defaultdict(float)
192         for size, _, ps, cs, _ in g.tensors:
193             producers = {mapping[u] for u in ps}
194             consumers = {mapping[u] for u in cs}
195             for a in producers:
196                 for b in consumers - {a}:
197                     affinity[a, b] += size
198         used = set()
199         accepted = 0
200         for (a, b), weight in sorted(affinity.items(), key=lambda x: (-x
201 [1], x[0])):
202             if a in used or b in used or len(groups[a]) + len(groups[b]) >
203 1024:
204                 continue
205             if getattr(g, 'fast_contractions', False):
206                 from q1_contraction import can_contract, contract
207                 if not can_contract(pred, succ, a, b):
208                     continue
209                 trial = {u: a if s == b else s for u, s in mapping.items()}
210             }
211             contract(pred, succ, a, b)
212         else:
213             trial = {u: a if s == b else s for u, s in mapping.items()}
214         }
215         try:
216             g.view(trial)
217         except ValueError:
218             continue
219         hierarchy.append((a, b, set(groups[b])))
220         mapping = trial
221         used.update((a, b))
222         accepted += 1
223         if accepted >= 32:
224             break
225     if not accepted:
226         break
227     _, current = g.schedule(mapping, cores)
228     # Reverse contractions ensure a later parent merge is undone before
229     # its child.
230     # If a parent stays merged, skip the child (its saved membership is no

```

```

    longer whole).
226 for a, b, members in reversed(hierarchy):
227     if any(mapping[u] != a for u in members):
228         continue
229     trial = dict(mapping)
230     for u in members:
231         trial[u] = b
232     try:
233         _, value = g.schedule(trial, cores)
234     except ValueError:
235         continue
236     if value < current:
237         mapping, current = trial, value
238 return mapping
239
240
241 def validate(g, plan):
242     from stub_multicore_cut_and_schedule import derive_multicore_plan
243     from evaluation_validation import validate_task_order
244     validate_task_order(derive_multicore_plan(g.raw, plan))
245
246
247 def mutate(g, plan, cores, rng, kind, preserve=False):
248     trial = copy.deepcopy(plan)
249     mapping = {int(u): s for u, s in plan['node_to_subgraph'].items()}
250     groups, pred, succ, order = g.view(mapping)
251     if not groups:
252         return trial
253     if kind == 'merge':
254         pairs = [(a, b) for a in sorted(succ) for b in sorted(succ[a])]
255         if not pairs:
256             return trial
257         a, b = rng.choice(pairs)
258         mapping = {u: a if s == b else s for u, s in mapping.items()}
259     elif kind == 'split':
260         choices = [s for s in sorted(groups) if len(groups[s]) > 1]
261         if not choices:
262             return trial
263         s = rng.choice(choices)
264         nodes = sorted(groups[s], key=g.pos.get)
265         cut = rng.randrange(1, len(nodes))
266         for u in nodes[cut:]:
267             mapping[u] = max(groups) + 1
268     elif kind == 'boundary':
269         edges = [(u, v) for u in g.order for v in sorted(g.succ[u]) if
mapping[u] != mapping[v]]
270         if not edges:
271             return trial
272         u, v = rng.choice(edges)
273         if rng.random() < .5:
274             mapping[u] = mapping[v]
275     else:

```

```

276         mapping[v] = mapping[u]
277     elif kind == 'migrate':
278         s = rng.choice(sorted(groups))
279         for seq in trial['core_schedules']:
280             if s in seq:
281                 seq.remove(s)
282                 target = trial['core_schedules'][rng.randrange(cores)]
283                 # Project a common topological ordering; official validation
284                 checks extra core edges.
285                 position = {v: i for i, v in enumerate(order)}
286                 target.append(s)
287                 target.sort(key=position.get)
288                 return trial
289     else:
290         choices = [seq for seq in trial['core_schedules'] if len(seq) > 1]
291         if choices:
292             seq = rng.choice(choices)
293             i = rng.randrange(len(seq) - 1)
294             seq[i], seq[i + 1] = seq[i + 1], seq[i]
295             return trial
296     if preserve == 'mobile':
297         from q1_guided_repair import guided_repair
298         return guided_repair(g, plan, mapping, cores)
299     if preserve:
300         from q1_partition import repair
301         return repair(g, plan, mapping)
302     return g.schedule(mapping, cores)[0]
303
304 def bottleneck_candidates(g, plan, result, cores):
305     """按真实 Task 持续时间选拆分对象，并尝试将长 Task 移到较早结束的核。"""
306     mapping = {int(u): s for u, s in plan['node_to_subgraph'].items()}
307     groups, _, _, order = g.view(mapping)
308     tasks = sorted((t for c in result['per_core_timeline'] for t in c['tasks']),
309                    key=lambda t: (-t['duration'], -t['end'], t['task_id']))
310     for task in tasks[:4]:
311         s = task['task_id']
312         nodes = sorted(groups[s], key=g.pos.get)
313         if len(nodes) < 2:
314             continue
315         # Balance the dominant compute Pipe, rather than splitting by op
316         count.
317         loads = defaultdict(int)
318         for u in nodes:
319             loads[g.ops[u]['pipe']] += g.ops[u]['cycles']
320         pipe = max(loads, key=lambda p: (loads[p], p))
321         partial = 0
322         cuts = []
323         for i, u in enumerate(nodes[:-1], 1):
324             if g.ops[u]['pipe'] == pipe:

```

```

324         partial += g.ops[u]['cycles']
325         cuts.append((abs.loads[pipe] / 2 - partial), i))
326     _, cut = min(cuts)
327     trial = dict(mapping)
328     for u in nodes[cut:]:
329         trial[u] = max(groups) + 1
330     try:
331         yield 'critical_split', g.schedule(trial, cores)[0]
332     except ValueError:
333         continue
334     finish = [max((t['end'] for t in c['tasks']), default=0) for c in
result['per_core_timeline']]
335     positions = {s: i for i, s in enumerate(order)}
336     for task in tasks[:3]:
337         s = task['task_id']
338         source = next(i for i, seq in enumerate(plan['core_schedules']) if
s in seq)
339         for target in sorted(range(cores), key=lambda c: (finish[c], c)):
340             if source == target:
341                 continue
342             trial = copy.deepcopy(plan)
343             trial['core_schedules'][source].remove(s)
344             trial['core_schedules'][target].append(s)
345             trial['core_schedules'][target].sort(key=positions.get)
346             yield 'critical_migrate', trial
347
348
349 def solve(g, cores=4, method='alns', budget=12, seed=0, block_size=64,
on_evaluation=None,
350         refine_budget=0):
351     """budget 是 ALNS 新候选的官方评估次数上限; 初解另计."""
352     from multicore_cut_evaluate_problem_1 import evaluate_scene_a
353     if cores < 1 or budget < 0 or block_size < 1 or refine_budget < 0:
354         raise ValueError('cores/block_size 必须为正, budget 不得为负')
355     if method not in {'greedy', 'multilevel', 'alns'}:
356         raise ValueError('未知方法')
357     rng = random.Random(seed)
358     cache, records, failures = {}, [], []
359     started = time.perf_counter()
360
361     def evaluate(plan, label):
362         key = json.dumps(plan, sort_keys=True)
363         if key in cache:
364             return cache[key], None
365         validate(g, plan)
366         t = time.perf_counter()
367         result = evaluate_scene_a(g.raw, plan, g.bandwidth, g.capacity, g.
cross, g.same)
368         score = (result['makespan'], result['data_movement_bytes']['
added_copy_bytes'])
369         # Full official traces can be large; retain only scores for
rejected candidates.

```

```

370         cache[key] = score
371         records.append({'candidate': label, 'makespan': score[0], '
added_copy_bytes': score[1],
372                        'evaluation_seconds': time.perf_counter() - t})
373         if on_evaluation is not None:
374             on_evaluation(records[-1])
375         return score, result
376
377     # Always keep the single-Task fallback: using available cores must not
force a regression.
378     fallback = {'node_to_subgraph': {str(u): 0 for u in sorted(g.ops)},
379                'core_schedules': [[0] if g.ops else []] + [[] for _ in
range(cores - 1)]}
380     best = fallback
381     best_score, best_result = evaluate(best, 'single_task')
382     single_time = best_score[0]
383     initial = greedy_partition(g, cores, block_size) if cores > 1 else {}
384     candidates = [('greedy', g.schedule(initial, cores)[0])] if cores > 1
else []
385     if cores > 1 and method in {'multilevel', 'alns'}:
386         candidates.append(('multilevel', g.schedule(multilevel(g, initial,
cores), cores)[0]))
387     for label, candidate in candidates:
388         score, result = evaluate(candidate, label)
389         if score < best_score:
390             best, best_score, best_result = candidate, score, result
391     current, current_score = best, best_score
392     kinds = ['merge', 'split', 'boundary', 'migrate', 'reorder']
393     weights = [1.0] * len(kinds)
394     evaluated = 0
395     if method == 'alns' and cores > 1:
396         for attempt in range(max(20, budget * 20)):
397             if evaluated >= budget:
398                 break
399             k = rng.choices(range(len(kinds)), weights=weights)[0]
400             kind = kinds[k]
401             try:
402                 trial = current
403                 # Compound destroy/repair of a small region, followed by
list-schedule repair.
404                 # Initial prototype uses 1--3 edits; not a whole-graph
random restart.
405                 for _ in range(rng.randint(1, 3)):
406                     trial = mutate(g, trial, cores, rng, kind)
407                     key = json.dumps(trial, sort_keys=True)
408                     if key in cache:
409                         continue
410                     validate(g, trial)
411             except (ValueError, RuntimeError) as exc:
412                 failures.append({'operation': kind, 'reason': str(exc)})
413                 continue
414     # Evaluator failures propagate: implementation bugs must not

```

```

Look like bad candidates.
415         score, result = evaluate(trial, kind)
416         evaluated += 1
417         improved = score < best_score
418         temp = max(1.0, single_time * .03 * (1 - evaluated / max(1,
budget)))
419         if score < current_score or rng.random() < math.exp(min(0, (
current_score[0] - score[0]) / temp)):
420             current, current_score = trial, score
421             if improved:
422                 best, best_score, best_result = trial, score, result
423                 weights[k] = .8 * weights[k] + .2 * (5 if improved else 1)
424             # Optional non-regressing refinement, separately budgeted for honest
comparisons.
425             refinement_evaluations = 0
426             if cores > 1 and refine_budget:
427                 from itertools import chain
428                 sizes = sorted({max(1, block_size // 4), max(1, block_size // 2),
block_size * 2})
429                 granular = ((f'granularity_{size}', g.schedule(greedy_partition(g,
cores, size), cores)[0]))
430                     for size in sizes)
431                 pending = chain(granular, bottleneck_candidates(g, best,
best_result, cores))
432                 for _ in range(refine_budget * 20):
433                     if refinement_evaluations >= refine_budget:
434                         break
435                     try:
436                         label, trial = next(pending)
437                     except StopIteration:
438                         break
439                     if json.dumps(trial, sort_keys=True) in cache:
440                         continue
441                     try:
442                         validate(g, trial)
443                     except (ValueError, RuntimeError) as exc:
444                         failures.append({'operation': label, 'reason': str(exc)})
445                         continue
446                     score, result = evaluate(trial, label)
447                     refinement_evaluations += 1
448                     if score < best_score:
449                         best, best_score, best_result = trial, score, result
450                         # Recompute the bottleneck after improvement; retain
untried granular candidates.
451                         pending = chain(pending, bottleneck_candidates(g, best,
best_result, cores))
452                 return best, best_result, {'method': method, 'seed': seed, 'budget':
budget,
453                     'refine_budget': refine_budget, 'refinement_evaluations':
refinement_evaluations,
454                     'block_size': block_size, 'singlecore_makespan': single_time,
455                     'speedup': single_time / best_score[0] if best_score[0] else

```

```

1.0,
456         'search_evaluations': evaluated, 'evaluations': records,
457         'rejected_proposals': failures, 'operator_weights': dict(zip(
kinds, weights)),
458         'elapsed_seconds': time.perf_counter() - started}

```

A.2 问题二求解器

程序 2 问题二求解器

```

1  """Scene B: communication placement, lifetime ordering, bounded joint
   repair.
2
3  All proxies generate candidates only. The unmodified B evaluator accepts
   them.
4  No tensor splitting, operation duplication, or replacement of official
   spilling.
5  """
6  from collections import defaultdict, Counter
7  import copy
8  import heapq
9  import random
10 import time
11 from q1_solver import Graph, greedy_partition, topological
12 from q2_evaluator import evaluate, key
13
14 VARIANTS = {'baseline': '', 'full': 'CLJ', 'no_C': 'LJ', 'no_L': 'CJ', '
   no_J': 'CL'}
15 DEFAULTS = {'block_sizes': (16, 64, 256), 'ready_window': 32,
16             'communication_weight': 1.0, 'lifetime_weight': 1.0,
17             'joint_pool': 12, 'joint_region': 4, 'proposal_cap': 24}
18
19
20 class SceneBGraph(Graph):
21     def __init__(self, raw, settings, delay):
22         # These attributes only parameterize inherited partition
construction.
23         # They are NOT scene-A task wait constraints in evaluation.
24         super().__init__(raw, settings, {'task_cross_core_wait_cycles':
   delay,
25                                         'task_same_core_wait_cycles': 0})
26         self.settings, self.delay = settings, delay
27         # Official B also materializes direct operation edges as UB
tensors.
28         # Keep these separate: unlike ordinary tensor fanout they are per-
edge.
29         self.direct_edges = [(e['source'], e['target'], max(0, int(e.get('
   data_size', 0))))
30                               for e in raw['edges']]
31         self.ops = [e for e in self.ops if e['source'] in self.ops and e['target'] in
   self.ops]
32
33

```

```

34 def owner_of(plan):
35     return {s: c for c, seq in enumerate(plan['core_schedules']) for s in
        seq}
36
37
38 def tensor_bytes(tensor, mapping, owner):
39     """Base scheduled bytes for one tensor; partial owner maps allowed.
40
41     Uses the official per producer-core/consumer-core COPY-pair convention
42     .
43     A tensor's final output copy is separate from its remote-consumer
44     copies.
45     """
46     size, _, ps, cs, final = tensor
47     pc = {owner[mapping[v]] for v in ps if mapping[v] in owner}
48     cc = {owner[mapping[v]] for v in cs if mapping[v] in owner}
49     incoming = len(cc) if not ps else 0
50     outgoing = len(pc) if final or not cs else 0
51     cross = sum(a != b for a in pc for b in cc)
52     return size * (incoming + outgoing + 2 * cross)
53
54 def base_bytes(g, plan):
55     mapping = {int(v): s for v, s in plan['node_to_subgraph'].items()}
56     owner = owner_of(plan)
57     return (sum(tensor_bytes(t, mapping, owner) for t in g.tensors)
58             + sum(2 * size for u,v,size in g.direct_edges
59                   if owner[mapping[u]] != owner[mapping[v]]))
60
61 def context(g, mapping):
62     groups, pred, succ, order = g.view(mapping)
63     incident, loads = {}, {}
64     for s, nodes in groups.items():
65         incident[s] = sorted({i for v in nodes for i in g.incident[v]})
66         load = defaultdict(int)
67         for v in sorted(nodes):
68             load[g.ops[v]['pipe']] += g.ops[v]['cycles']
69         loads[s] = dict(load)
70     ranks = {}
71     for s in reversed(order):
72         ranks[s] = max.loads[s].values(), default=0) + max((ranks[v] for v
73         in succ[s]), default=0)
74     return groups, pred, succ, order, incident, loads, ranks
75
76 def place(g, mapping, cores, communication=True, weight=1.0):
77     """Topological list placement using Pipe load and de-duplicated
78     marginal bytes."""
79     groups, pred, succ, _, incident, loads, ranks = context(g, mapping)
80     order = topological(pred, succ, ranks)
81     direct = defaultdict(list)

```

```

81     for u,v,size in g.direct_edges:
82         for sg in {mapping[u],mapping[v]}: direct[sg].append((u,v,size))
83     def edge_bytes(s, assigned):
84         return sum(2*size for u,v,size in direct[s]
85                     if mapping[u] in assigned and mapping[v] in assigned
86                     and assigned[mapping[u]] != assigned[mapping[v]])
87     owners, end = {}, {}
88     core_load = [defaultdict(float) for _ in range(cores)]
89     for s in order:
90         before = sum(tensor_bytes(g.tensors[i], mapping, owners) for i in
91 incident[s]) + edge_bytes(s,owners)
92         options = []
93         for c in range(cores):
94             trial = dict(owners); trial[s] = c
95             delta = sum(tensor_bytes(g.tensors[i], mapping, trial) for i
96 in incident[s]) + edge_bytes(s,trial) - before
97             work = max((core_load[c][p] + loads[s].get(p, 0)
98                         for p in set(core_load[c]) | set(loads[s])),
99 default=0)
100             release = max((end[p] + (g.delay if owners[p] != c else 0)
101                             for p in pred[s]), default=0)
102             finish = max(work, release + max(loads[s].values(), default=0)
103 )
104             score = finish + (weight * delta / g.bandwidth if
105 communication else 0)
106             options.append((score, finish, c))
107             _, finish, c = min(options)
108             owners[s], end[s] = c, finish
109             for p, amount in loads[s].items():
110                 core_load[c][p] += amount
111         return owners
112
113 def order_plan(g, mapping, owners, cores, lifetime=True, window=32, weight
114 =1.0,
115               random_priority=None):
116     """Project ONE global legal group order to cores; no core-graph
117 contraction.
118
119 Lifetime is a group-level no-spill proxy. External COPY_OUT timing and
120 Pipe
121 overlap are resolved only by the official evaluator.
122 """
123     groups, pred, succ, _, incident, loads, ranks = context(g, mapping)
124     counts = Counter()
125     for s in groups:
126         for i in incident[s]:
127             counts[owners[s], i] += 1
128     live = [set() for _ in range(cores)]
129     resident = [defaultdict(int) for _ in range(cores)]
130     degree = {s: len(pred[s]) for s in groups}
131     priority = random_priority or ranks

```

```

125 ready = [(-priority[s], s) for s in groups if degree[s] == 0]
126 heapq.heapify(ready)
127 schedules = [[] for _ in range(cores)]
128 rank_scale = max(ranks.values(), default=1) or 1
129 while ready:
130     candidates = [heapq.heappop(ready) for _ in range(min(window if
lifetime else 1, len(ready)))]
131     def score(item):
132         _, s = item; c = owners[s]
133         added, freed = defaultdict(int), defaultdict(int)
134         for i in incident[s]:
135             size, tier, *_ = g.tensors[i]
136             if i not in live[c]: added[tier] += size
137             if counts[c, i] == 1: freed[tier] += size
138             pressure = sum(max(0, resident[c][r] + added[r] - cap) / cap
139                             for r, cap in g.capacity.items())
140             delta = sum((added[r] - freed[r]) / cap for r, cap in g.
capacity.items())
141             return (weight * (pressure + delta) - ranks[s] / rank_scale, -
ranks[s], s)
142     chosen = min(candidates, key=score) if lifetime else candidates[0]
143     for item in candidates:
144         if item != chosen: heapq.heappush(ready, item)
145     s = chosen[1]; c = owners[s]; schedules[c].append(s)
146     for i in incident[s]:
147         size, tier, *_ = g.tensors[i]
148         if i not in live[c]: live[c].add(i); resident[c][tier] += size
149         counts[c, i] -= 1
150         if counts[c, i] == 0: live[c].remove(i); resident[c][tier] -=
size
151     for v in sorted(succ[s]):
152         degree[v] -= 1
153         if degree[v] == 0: heapq.heappush(ready, (-priority[v], v))
154     if sum(map(len, schedules)) != len(groups): raise ValueError('Cyclic
group order')
155     return {'node_to_subgraph': {str(v): mapping[v] for v in sorted(
mapping)},
156           'core_schedules': schedules}
157
158
159 def proxy(g, plan):
160     """Cycles-scaled ranking only; neither exact time nor a feasibility
proof."""
161     mapping = {int(v): s for v, s in plan['node_to_subgraph'].items()}
162     owners = owner_of(plan)
163     loads = [defaultdict(int) for _ in plan['core_schedules']]
164     positions = {}
165     groups = defaultdict(list)
166     for v in g.order: groups[mapping[v]].append(v)
167     for c, seq in enumerate(plan['core_schedules']):
168         n = 0
169         for s in seq:

```

```

170         for v in groups[s]:
171             positions[v] = n; n += 1
172             loads[c][g.ops[v]['pipe']] += g.ops[v]['cycles']
173     events = [defaultdict(lambda: defaultdict(int)) for _ in loads]
174     for size, tier, ps, cs, _ in g.tensors:
175         uses = defaultdict(list)
176         for v in sorted(ps | cs): uses[owners[mapping[v]]].append(
positions[v])
177         for c, pos in uses.items():
178             events[c][min(pos)][tier] += size
179             events[c][max(pos) + 1][tier] -= size
180     excess = 0
181     for changes in events:
182         live, peak = defaultdict(int), defaultdict(int)
183         for j in sorted(changes):
184             for tier, d in changes[j].items():
185                 live[tier] += d; peak[tier] = max(peak[tier], live[tier])
186         excess += sum(max(0, peak[r] - cap) for r, cap in g.capacity.items
())
187     traffic = base_bytes(g, plan)
188     work = max((max(d.values(), default=0) for d in loads), default=0)
189     return max(work, traffic / g.bandwidth) + 2 * excess / g.bandwidth
190
191
192 def proposal(g, plan, cores, rng, features, cfg, joint=False):
193     mapping = {int(v): s for v, s in plan['node_to_subgraph'].items()}
194     owners = owner_of(plan)
195     groups, pred, succ, order, incident, loads, ranks = context(g, mapping
)
196     if not groups: return copy.deepcopy(plan), {'move': 'empty'}
197     if joint:
198         # Inspect at most a bounded set of high criticality /
communication groups.
199         ranked = sorted(groups, key=lambda s: (-(ranks[s] + sum(g.tensors[
i][0] for i in incident[s]) / g.bandwidth), s))
200         offset = rng.randrange(max(1, len(ranked)))
201         region = list(dict.fromkeys(ranked[:max(1, cfg['joint_region']//2)
] + ranked[offset:offset+cfg['joint_region']]))[:cfg['joint_region']]
202         choices = [(s,c) for s in region for c in range(cores) if owners[s
] != c]
203         rng.shuffle(choices)
204         candidates = []
205         for s,c in choices[:cfg['joint_pool']]:
206             target = dict(owners); target[s] = c
207             trial = order_plan(g,mapping,target,cores,'L' in features,cfg[
'ready_window'],cfg['lifetime_weight'])
208             candidates.append((proxy(g,trial),key(trial),trial,{'move':'
joint_migrate_order','subgraph':s,'core':c}))
209         if candidates:
210             _,_,trial,info = min(candidates, key=lambda x:x[:2]); return
trial,info
211         kind = rng.choice(('migrate','order','split','boundary'))

```

```

212     if kind == 'migrate':
213         s = rng.choice(sorted(groups)); owners[s] = rng.randrange(cores)
214     elif kind == 'split':
215         choices = [s for s in sorted(groups) if len(groups[s]) > 1]
216         if choices:
217             s = rng.choice(choices); nodes = sorted(groups[s],key=g.pos.
get)
218             cut = len(nodes)//2; new = max(groups)+1
219             for v in nodes[cut:]: mapping[v] = new
220             owners[new] = owners[s]
221     elif kind == 'boundary':
222         edges = [(u,v) for u in g.order for v in sorted(g.succ[u]) if
mapping[u]!=mapping[v]]
223         if edges:
224             u,v = rng.choice(edges); mapping[u] = mapping[v]
225             valid_groups = set(mapping.values()); owners = {s:c for s,c in
owners.items() if s in valid_groups}
226         # Splits or boundary moves must recheck quotient acyclicity before
ranking.
227         _,p,s,_,_,r = context(g,mapping)
228         random_priority = {v:rng.random() for v in p} if kind=='order' else
None
229         trial = order_plan(g,mapping,owners,cores,'L' in features and kind!='
order',
230                             cfg['ready_window'],cfg['lifetime_weight'],
random_priority)
231         return trial,{'move':kind}
232
233
234 def solve(raw, settings, delay, cores, budget=12, seed=0, variant='full',
params=None,
235          on_evaluation=None):
236     if cores not in range(1,6) or budget < 1: raise ValueError('Invalid
cores/budget')
237     features = VARIANTS[variant]
238     cfg = {**DEFAULTS, **(params or {})}
239     if set(cfg) != set(DEFAULTS): raise ValueError('Unknown parameter')
240     if any(cfg[k] < 1 for k in ('ready_window','joint_pool','joint_region'
,'proposal_cap')):
241         raise ValueError('Window/pool/region/proposal limits must be
positive')
242     if not cfg['block_sizes'] or any(x < 1 for x in cfg['block_sizes']):
243         raise ValueError('Block sizes must be positive')
244     if any(cfg[k] < 0 for k in ('communication_weight','lifetime_weight')):
245         raise ValueError('Weights must be nonnegative')
246     rng = random.Random(seed)
247     g = SceneBGraph(raw,settings,delay)
248     rows, generation_errors = [], []
249     best_plan = best_result = None
250     seen = set()
251     # Cache only compact failures; full success outputs are not

```

```

accumulated in RAM.
252     def assess(plan, label, info=None):
253         nonlocal best_plan, best_result
254         started=time.perf_counter(); fingerprint=key(plan)
255         row={'candidate':label, 'plan_sha256':fingerprint, 'duplicate':
fingerprint in seen,
256             'mechanism':info or {}}
257         seen.add(fingerprint)
258         try:
259             result=evaluate(raw, plan, settings, delay)
260             objective=(result['makespan'], result['data_movement_bytes'] ['
added_copy_bytes'])
261             row.update(status='success', makespan=objective[0],
added_copy_bytes=objective[1],
262                       partition_added_copy_bytes=result['
data_movement_bytes'] ['partition_added_copy_bytes'],
263                       spill_added_copy_bytes=result['data_movement_bytes'
'] ['spill_added_copy_bytes'])
264             if best_result is None or objective < (best_result['makespan'
'], best_result['data_movement_bytes'] ['added_copy_bytes']):
265                 best_plan, best_result=copy.deepcopy(plan), result; row['
accepted']=True
266             else: row['accepted']=False
267             except (ValueError, RuntimeError) as exc:
268                 row.update(status='error', error_type=type(exc).__name__, error=
str(exc))
269                 row['seconds']=time.perf_counter()-started; rows.append(row)
270                 if on_evaluation: on_evaluation(row)
271                 whole={'node_to_subgraph':{str(v):0 for v in sorted(g.ops)}, '
core_schedules':[[0] if g.ops else []]+[[[] for _ in range(cores-1)]]
272                 assess(whole, 'whole_graph')
273                 if cores>1:
274                     for size in cfg['block_sizes']:
275                         if len(rows)>=budget: break
276                         mapping=greedy_partition(g, cores, size)
277                         owners=place(g, mapping, cores, 'C' in features, cfg['
communication_weight'])
278                         plan=order_plan(g, mapping, owners, cores, 'L' in features, cfg['
ready_window'], cfg['lifetime_weight'])
279                         assess(plan, 'initial_'+str(size))
280                     while len(rows)<budget:
281                         if best_plan is None: break
282                         trial=None; info={}
283                         for attempt in range(cfg['proposal_cap']):
284                             try:
285                                 trial, info=proposal(g, best_plan, cores, rng, features, cfg
, joint=('J' in features and len(rows)%2==0))
286                                 if key(trial) not in seen: break
287                             except ValueError as exc:
288                                 generation_errors.append({'evaluation_slot':len(rows),
'attempt':attempt, 'error':str(exc)})
289                                 trial=None

```

```

290         if trial is None: trial=copy.deepcopy(best_plan);info={'move':
'bounded_fallback'}
291         assess(trial,'repair_'+str(len(rows)),info)
292         if best_result is None: raise RuntimeError('No scene-B feasible
candidate; evaluation errors: '+str(rows))
293         return best_plan,best_result,{ 'variant':variant,'features':list(
features),'seed':seed,'budget':budget,
294                                     'parameters':cfg,'evaluations':rows,'
evaluation_count':len(rows),
295                                     'official_calls':len(rows),'
generation_errors':generation_errors}

```

A.3 问题三求解器

程序 3 问题三求解器

```

1  """Q2-seeded deterministic cache-event neighborhood search; official Q3
   scoring."""
2  import copy
3  import importlib
4  import time
5  from collections import Counter, OrderedDict
6  from q1_io import PROCESSED
7  from q2_evaluator import check, key, load as load_b
8  from q2_solver import SceneBGraph
9  from q3_neighborhood import pool
10 from q2_physical import PhysicalScorer
11
12
13 def load():
14     settings, delay, provenance = load_b()
15     module = importlib.import_module('multicore_cut_evaluate_problem_3')
16     cache = module.read_cache_config(str(PROCESSED / 'data/config.txt'))
17     return settings, delay, cache, provenance
18
19
20 def evaluate(raw, plan, settings, delay, cache):
21     from multicore_cut_evaluate_problem_3 import evaluate_problem_3
22     r = evaluate_problem_3(raw, plan, settings['bandwidth'], settings['
capacity'], delay,
23                             cache['cache_capacity_bytes'], cache['
cache_bandwidth_bytes_per_cycle'])
24     check(r, settings, delay)
25     return r
26
27
28 def audit_cache(r):
29     """Independent FIFO ledger from exported ordered events, including
reinsertion."""
30     q = OrderedDict(); used = 0; hits = misses = nh = nm = peak = 0
31     for e in r['cache_events']:
32         t, size = e['tensor_id'], e['size_bytes']
33         if e['event'] in ('hit', 'miss'):

```

```

34         hit = t in q
35         assert hit == (e['event'] == 'hit')
36         if hit: hits += size; nh += 1
37         else: misses += size; nm += 1
38     else:
39         assert e['event'] == 'insert' and t not in q and size <= r['
cache_capacity_bytes']
40         evicted = []
41         while q and used + size > r['cache_capacity_bytes']:
42             old, s = q.popitem(last=False); used -= s; evicted.append(
old)
43         q[t] = size; used += size; peak = max(peak, used)
44         assert evicted == e['evicted_tensor_ids'] and used == e['
used_bytes']
45         s = r['cache_stats']
46         assert (hits, misses, nh, nm) == (s['hit_bytes'], s['miss_bytes'], s['
copy_in_hits'], s['copy_in_misses'])
47         assert s['hit_rate'] == (hits/(hits+misses) if hits+misses else 0)
48         assert used == r['cache_used_bytes_final'] and peak <= r['
cache_capacity_bytes']
49         assert list(q.items()) == [(e['tensor_id'], e['size_bytes']) for e in
r['cache_final_entries']]
50         paths = Counter(o.get('memory_path') for c in r['per_core_timeline']
for o in c['ops'] if o['op'] in ('COPY_IN', 'COPY_OUT'))
51         assert not paths['ON_CHIP'], 'DDR-derived metric requires no on-chip
COPY'
52         return dict(fifo_ledger=True, peak_bytes=peak, hit_bytes=hits,
miss_bytes=misses,
53                     physical_ddr_bytes_derived=r['data_movement_bytes']['
scheduled_copy_bytes']-hits)
54
55
56 def audit(raw, plan, r, settings, delay):
57     cache = audit_cache(r)
58     profiles = PhysicalScorer(raw, settings, delay).actual_lifetimes(plan,
r)
59     return dict(cache=cache, global_memory_peak={str(c):p['peak_bytes']
for c,p in profiles.items()},
60               synchronization=True, traffic_identity=True)
61
62
63 def cache_diagnostic(r):
64     """Prioritize groups with repeated actual misses; no claimed causal
savings."""
65     events = [e for e in r['cache_events'] if e['event']=='miss']
66     counts = Counter(e['tensor_id'] for e in events)
67     groups = {(c['core_id'],o['op_id']):o['subgraph_id'] for c in r['
per_core_timeline'] for o in c['ops']}
68     score = Counter()
69     for e in events:
70         g = groups[e['core_id'],e['op_id']]
71         if g is not None and counts[e['tensor_id']]>1: score[g] += e['

```

```

size_bytes']*(counts[e['tensor_id']]-1)
72     return dict(group_priority=sorted(score,key=lambda g:(-score[g],g)),
    hol=[], group_scores=dict(score))
73
74
75 def solve(raw, seed, settings, delay, cache, budget=8, mode='cache'):
76     start=time.perf_counter(); anchor=evaluate(raw,seed,settings,delay,
    cache)
77     best=copy.deepcopy(seed); result=anchor; rows=[]; seen={key(seed)}
78     g=SceneBGraph(raw,settings,delay)
79     diagnostic=cache_diagnostic(anchor) if mode=='cache' else None
80     proposals=pool(g,seed,diagnostic,budget=budget) if budget else []
81     for item in proposals:
82         p=item['plan']; h=key(p)
83         if h in seen: continue
84         seen.add(h); t=time.perf_counter()
85         try:
86             r=evaluate(raw,p,settings,delay,cache)
87             accept=(r['makespan'],r['data_movement_bytes'] ['
    added_copy_bytes']) < (result['makespan'],result['data_movement_bytes'
    ] ['added_copy_bytes'])
88             if accept: best,result=copy.deepcopy(p),r
89             rows.append(dict(status='ok',plan_sha256=h,move=item['move'],
    makespan=r['makespan'],added_copy_bytes=r['data_movement_bytes'] ['
    added_copy_bytes'],hit_rate=r['cache_stats'] ['hit_rate'],accepted=
    accept,seconds=time.perf_counter()-t))
90             except (RuntimeError,ValueError) as exc:
91                 rows.append(dict(status='invalid',plan_sha256=h,move=item['
    move'],error=str(exc),seconds=time.perf_counter()-t))
92             search_seconds=time.perf_counter()-start
93             replay=evaluate(raw,best,settings,delay,cache)
94             assert replay==result
95             checks=audit(raw,best,replay,settings,delay)
96             return best,replay,anchor,dict(mode=mode,budget=budget,candidate_calls
    =len(rows),official_calls=len(rows)+2,
97                 search_seconds=search_seconds,total_seconds=time.perf_counter()-
    start,seed_plan_sha256=key(seed),
98                 selected_plan_sha256=key(best),replay_equal=True,checks=checks,
    evaluations=rows,diagnostic=diagnostic)

```

B 问题一逐用例官方结果

每行对应一张图和一个核心数，共 500 行；加速比的分母为该图固定单核基准。额外搬运量为官方评价输出的逻辑字节数。

表 7 问题一全部算例的官方评价结果

算例	核数	Makespan	额外搬运量	加速比
case_001	1	233110	0	1.000000
case_001	2	116868	1152	1.994644
case_001	3	78545	2304	2.967853
case_001	4	58984	3456	3.952089
case_001	5	47502	4608	4.907372
case_002	1	261945	0	1.000000
case_002	2	138077	477696	1.897094
case_002	3	95931	532992	2.730556
case_002	4	74618	536064	3.510480
case_002	5	60651	536064	4.318890
case_003	1	602212	0	1.000000
case_003	2	462800	8565562	1.301236
case_003	3	378576	9989392	1.590729
case_003	4	329107	11891502	1.829836
case_003	5	294109	11891502	2.047581
case_004	1	392357	0	1.000000
case_004	2	199074	110592	1.970910
case_004	3	134043	221184	2.927098
case_004	4	104188	331776	3.765856
case_004	5	88341	442368	4.441392
case_005	1	95618	0	1.000000
case_005	2	91571	2718104	1.044195
case_005	3	88392	2736536	1.081749
case_005	4	87624	2736440	1.091231
case_005	5	87840	2736440	1.088547
case_006	1	82837	16400	1.000000
case_006	2	42379	56200	1.954671
case_006	3	32925	337092	2.515930
case_006	4	31129	533902	2.661088
case_006	5	31729	91264	2.610766
case_007	1	1012562	0	1.000000
case_007	2	507494	27648	1.995220
case_007	3	338094	55296	2.994913
case_007	4	254398	82944	3.980228
case_007	5	203834	110592	4.967581
case_008	1	487605	0	1.000000
case_008	2	244266	0	1.996205
case_008	3	163359	0	2.984868
case_008	4	123060	0	3.962335
case_008	5	100603	0	4.846824
case_009	1	153261	0	1.000000
case_009	2	92259	182950	1.661204

算例	核数	Makespan	额外搬运量	加速比
case_009	3	92004	193702	1.665808
case_009	4	92192	225586	1.662411
case_009	5	91903	214604	1.667639
case_010	1	86969	0	1.000000
case_010	2	45626	48384	1.906128
case_010	3	34518	395696	2.519526
case_010	4	30262	575880	2.873868
case_010	5	28842	958646	3.015360
case_011	1	184697	0	1.000000
case_011	2	92646	180224	1.993578
case_011	3	69625	1081344	2.652740
case_011	4	46880	540672	3.939782
case_011	5	47047	1630208	3.925798
case_012	1	59088	0	1.000000
case_012	2	29489	71816	2.003730
case_012	3	20230	43320	2.920811
case_012	4	15724	86400	3.757822
case_012	5	13406	53568	4.407579
case_013	1	263949	0	1.000000
case_013	2	132198	8192	1.996619
case_013	3	88351	16384	2.987504
case_013	4	66982	24576	3.940596
case_013	5	54363	32768	4.855306
case_014	1	17698626	71010720	1.000000
case_014	2	8909568	69998520	1.986474
case_014	3	6002519	84088488	2.948533
case_014	4	4543360	76510104	3.895493
case_014	5	3694137	88763688	4.791004
case_015	1	177362	180256	1.000000
case_015	2	90480	199944	1.960234
case_015	3	61018	298252	2.906716
case_015	4	48866	365824	3.629558
case_015	5	44068	331776	4.024735
case_016	1	7715523	0	1.000000
case_016	2	5440637	134500220	1.418129
case_016	3	4478897	134500220	1.722639
case_016	4	4266542	134500220	1.808379
case_016	5	4158373	134500220	1.855419
case_017	1	162810	22528	1.000000
case_017	2	81117	110100	2.007101
case_017	3	53799	196920	3.026264
case_017	4	41101	285984	3.961217
case_017	5	33081	359460	4.921556
case_018	1	1004367	1970176	1.000000
case_018	2	501235	2622524	2.003785
case_018	3	336388	4077600	2.985740
case_018	4	253676	4938752	3.959251

算例	核数	Makespan	额外搬运量	加速比
case_018	5	203806	5603348	4.928054
case_019	1	81144	0	1.000000
case_019	2	40136	42540	2.021726
case_019	3	27848	83520	2.913818
case_019	4	24465	285312	3.316738
case_019	5	23450	568740	3.460299
case_020	1	532746	0	1.000000
case_020	2	266580	0	1.998447
case_020	3	177950	0	2.993796
case_020	4	133704	0	3.984518
case_020	5	107650	0	4.948871
case_021	1	6567937	4501504	1.000000
case_021	2	3168548	50104320	2.072854
case_021	3	2124156	50071552	3.092022
case_021	4	1550324	4608000	4.236493
case_021	5	1249916	11845632	5.254703
case_022	1	56780	0	1.000000
case_022	2	28303	6912	2.006148
case_022	3	18978	13824	2.991885
case_022	4	14336	20736	3.960658
case_022	5	11479	24612	4.946424
case_023	1	122614	0	1.000000
case_023	2	61065	73728	2.007926
case_023	3	40682	207264	3.013962
case_023	4	30516	281280	4.018023
case_023	5	24940	280992	4.916359
case_024	1	2555343	0	1.000000
case_024	2	1803647	44243528	1.416764
case_024	3	1483888	44243528	1.722059
case_024	4	1413568	44243528	1.807726
case_024	5	1378992	44243528	1.853051
case_025	1	4863026	0	1.000000
case_025	2	2431840	27648	1.999731
case_025	3	1622164	55296	2.997863
case_025	4	1217444	82944	3.994456
case_025	5	975282	110592	4.986277
case_026	1	99203	0	1.000000
case_026	2	51839	218580	1.913675
case_026	3	42174	381752	2.352231
case_026	4	42089	217872	2.356982
case_026	5	43946	414504	2.257384
case_027	1	2363952	1400832	1.000000
case_027	2	1116984	1843200	2.116371
case_027	3	804739	19496960	2.937539
case_027	4	559196	3317760	4.227412
case_027	5	558753	5529600	4.230764
case_028	1	43836040	277412256	1.000000

算例	核数	Makespan	额外搬运量	加速比
case_028	2	20808183	310682880	2.106673
case_028	3	13957556	310848768	3.140667
case_028	4	10676362	310848768	4.105897
case_028	5	9004662	286344264	4.868149
case_029	1	37555	0	1.000000
case_029	2	19740	0	1.902482
case_029	3	12743	0	2.947108
case_029	4	11028	0	3.405423
case_029	5	9383	0	4.002451
case_030	1	2808034	5519360	1.000000
case_030	2	1399339	5285020	2.006686
case_030	3	932367	8365156	3.011726
case_030	4	700342	10404864	4.009518
case_030	5	560306	11168828	5.011608
case_031	1	1414369	3491328	1.000000
case_031	2	712262	5250048	1.985743
case_031	3	473822	7544832	2.985022
case_031	4	356417	7926272	3.968298
case_031	5	284848	7931904	4.965346
case_032	1	41443	0	1.000000
case_032	2	20900	96344	1.982919
case_032	3	14009	96406	2.958313
case_032	4	10933	63656	3.790634
case_032	5	10280	80036	4.031420
case_033	1	1774196	4592640	1.000000
case_033	2	888544	6312520	1.996745
case_033	3	590697	7602264	3.003564
case_033	4	443447	7214688	4.000920
case_033	5	353592	8227976	5.017636
case_034	1	842996	1079832	1.000000
case_034	2	419167	1915392	2.011122
case_034	3	278067	2762240	3.031629
case_034	4	205314	3207748	4.105887
case_034	5	167930	3885104	5.019925
case_035	1	169571	0	1.000000
case_035	2	95406	0	1.777362
case_035	3	95782	42278	1.770385
case_035	4	94869	47410	1.787423
case_035	5	95983	83532	1.766677
case_036	1	931510	0	1.000000
case_036	2	466068	1152	1.998657
case_036	3	311345	2304	2.991890
case_036	4	233584	3456	3.987902
case_036	5	187182	4608	4.976493
case_037	1	210001	0	1.000000
case_037	2	105374	204800	1.992911
case_037	3	74038	1064960	2.836395

算例	核数	Makespan	额外搬运量	加速比
case_037	4	53320	614400	3.938503
case_037	5	43800	1925120	4.794543
case_038	1	1364555	5627904	1.000000
case_038	2	674500	6790656	2.023062
case_038	3	450730	7879736	3.027433
case_038	4	338163	8930344	4.035199
case_038	5	274658	9635364	4.968197
case_039	1	1435262	7929856	1.000000
case_039	2	708557	11550720	2.025613
case_039	3	482318	15020032	2.975759
case_039	4	395323	17604608	3.630606
case_039	5	431161	20480000	3.328831
case_040	1	253454	0	1.000000
case_040	2	120926	283200	2.095943
case_040	3	109777	642836	2.308808
case_040	4	109607	1022796	2.312389
case_040	5	108131	727212	2.343953
case_041	1	6003957	30898176	1.000000
case_041	2	2990398	33669120	2.007745
case_041	3	1991911	34851840	3.014169
case_041	4	1490603	39164928	4.027871
case_041	5	1201884	40673280	4.995455
case_042	1	1686740	6629376	1.000000
case_042	2	845091	6614304	1.995927
case_042	3	559018	8452672	3.017327
case_042	4	419432	8676192	4.021486
case_042	5	336330	10425184	5.015134
case_043	1	677656	49160	1.000000
case_043	2	353710	589446	1.915852
case_043	3	267112	2381570	2.536973
case_043	4	266590	2584974	2.541941
case_043	5	264401	2875646	2.562986
case_044	1	154407	4438112	1.000000
case_044	2	119062	5274528	1.296862
case_044	3	123085	5678624	1.254475
case_044	4	120120	5792192	1.285440
case_044	5	131771	6309600	1.171783
case_045	1	125614	0	1.000000
case_045	2	63059	0	1.992007
case_045	3	42601	0	2.948616
case_045	4	31878	0	3.940461
case_045	5	26164	0	4.801024
case_046	1	276455	3176288	1.000000
case_046	2	152419	5255872	1.813783
case_046	3	128115	4762304	2.157866
case_046	4	116069	5013664	2.381816
case_046	5	111389	4672800	2.481888

算例	核数	Makespan	额外搬运量	加速比
case_047	1	326508	0	1.000000
case_047	2	276535	4380996	1.180711
case_047	3	241969	5051902	1.349379
case_047	4	225654	5078720	1.446941
case_047	5	222583	5064318	1.466904
case_048	1	222220	0	1.000000
case_048	2	197496	1716280	1.125187
case_048	3	197613	1716280	1.124521
case_048	4	197613	1716280	1.124521
case_048	5	197613	1716280	1.124521
case_049	1	196276	0	1.000000
case_049	2	156968	2635492	1.250420
case_049	3	132866	4012736	1.477248
case_049	4	123251	4012736	1.592490
case_049	5	117489	4013760	1.670590
case_050	1	149690	0	1.000000
case_050	2	108323	51814	1.381886
case_050	3	104366	3416202	1.434279
case_050	4	98003	3448972	1.527402
case_050	5	99382	3424394	1.506208
case_051	1	607628	0	1.000000
case_051	2	429872	10225204	1.413509
case_051	3	352843	10225204	1.722092
case_051	4	336378	10225204	1.806384
case_051	5	328989	10225204	1.846955
case_052	1	180301	0	1.000000
case_052	2	90903	102912	1.983444
case_052	3	60160	181248	2.997025
case_052	4	45777	254976	3.938681
case_052	5	37846	333312	4.764070
case_053	1	1472686	5405536	1.000000
case_053	2	774757	5959552	1.900836
case_053	3	509253	6480220	2.891855
case_053	4	491606	10892964	2.995663
case_053	5	474708	20272004	3.102299
case_054	1	815051	0	1.000000
case_054	2	510200	6569360	1.597513
case_054	3	402111	6732340	2.026930
case_054	4	362394	11175330	2.249074
case_054	5	333071	11208098	2.447079
case_055	1	118835	69632	1.000000
case_055	2	56902	176664	2.088415
case_055	3	38966	168968	3.049710
case_055	4	32118	173568	3.699950
case_055	5	24831	280068	4.785752
case_056	1	277285	0	1.000000
case_056	2	231882	4095676	1.195802

算例	核数	Makespan	额外搬运量	加速比
case_056	3	199699	6006214	1.388515
case_056	4	189615	6006214	1.462358
case_056	5	186638	6006214	1.485684
case_057	1	59726	94216	1.000000
case_057	2	30047	182060	1.987753
case_057	3	20691	18228	2.886569
case_057	4	16471	32328	3.626131
case_057	5	14319	189060	4.171101
case_058	1	4539717	28696576	1.000000
case_058	2	2268326	33972224	2.001351
case_058	3	1532367	31514624	2.962552
case_058	4	1152464	42893312	3.939140
case_058	5	1002440	33316864	4.528667
case_059	1	829473	1691648	1.000000
case_059	2	369318	360448	2.245959
case_059	3	254823	4196352	3.255095
case_059	4	185216	1081344	4.478409
case_059	5	150782	4325376	5.501141
case_060	1	888789	2068520	1.000000
case_060	2	441525	2746508	2.012998
case_060	3	295983	3159296	3.002838
case_060	4	223107	3960200	3.983689
case_060	5	180081	3733888	4.935496
case_061	1	77191	0	1.000000
case_061	2	39298	34560	1.964248
case_061	3	28342	43008	2.723555
case_061	4	24024	70656	3.213079
case_061	5	21941	323072	3.518117
case_062	1	2854809	0	1.000000
case_062	2	1463796	3431424	1.950278
case_062	3	982412	3469824	2.905918
case_062	4	744188	3511296	3.836140
case_062	5	605685	3469824	4.713356
case_063	1	1070397	0	1.000000
case_063	2	552512	1903104	1.937328
case_063	3	371366	1903104	2.882324
case_063	4	283231	1915392	3.779237
case_063	5	229053	1920000	4.673141
case_064	1	19116	0	1.000000
case_064	2	19089	17994	1.001414
case_064	3	19089	17994	1.001414
case_064	4	19076	15692	1.002097
case_064	5	19083	7366	1.001729
case_065	1	53068	0	1.000000
case_065	2	26410	13984	2.009390
case_065	3	17964	27936	2.954130
case_065	4	13800	41904	3.845507

算例	核数	Makespan	额外搬运量	加速比
case_065	5	10949	66132	4.846835
case_066	1	359070	0	1.000000
case_066	2	242005	4846614	1.483730
case_066	3	176353	4877334	2.036087
case_066	4	162001	5725374	2.216468
case_066	5	140194	5725374	2.561237
case_067	1	63834834	264449408	1.000000
case_067	2	32203165	262601984	1.982253
case_067	3	21585693	247821184	2.957275
case_067	4	16358776	262601984	3.902177
case_067	5	13455316	262606080	4.744209
case_068	1	362095	0	1.000000
case_068	2	243395	4401532	1.487685
case_068	3	224168	4401532	1.615284
case_068	4	225518	4426108	1.605615
case_068	5	226325	4426684	1.599890
case_069	1	25144	0	1.000000
case_069	2	21862	514378	1.150124
case_069	3	21862	514378	1.150124
case_069	4	21862	514378	1.150124
case_069	5	24830	71044	1.012646
case_070	1	117300	0	1.000000
case_070	2	58498	129536	2.005197
case_070	3	39333	259072	2.982229
case_070	4	30080	333312	3.899601
case_070	5	25374	449024	4.622842
case_071	1	18919	0	1.000000
case_071	2	18901	93316	1.000952
case_071	3	18757	57220	1.008637
case_071	4	18449	155940	1.025476
case_071	5	18757	57220	1.008637
case_072	1	23897828	108613680	1.000000
case_072	2	11841342	110234400	2.018169
case_072	3	7929522	115280672	3.013779
case_072	4	5990048	120472240	3.989589
case_072	5	4830220	127451824	4.947565
case_073	1	10081626	141570432	1.000000
case_073	2	5168597	124388224	1.950554
case_073	3	3836078	131784832	2.628108
case_073	4	3249411	131636352	3.102601
case_073	5	3017992	124387200	3.340508
case_074	1	1682239	6070280	1.000000
case_074	2	831774	6869040	2.022471
case_074	3	551928	7438392	3.047932
case_074	4	414717	8323084	4.056354
case_074	5	335011	8036364	5.021444
case_075	1	1441809	0	1.000000

算例	核数	Makespan	额外搬运量	加速比
case_075	2	1008737	12444746	1.429321
case_075	3	778382	12461130	1.852315
case_075	4	698816	12461130	2.063217
case_075	5	709725	12444746	2.031504
case_076	1	6185113	28280832	1.000000
case_076	2	3065214	31095808	2.017841
case_076	3	2050028	34332672	3.017087
case_076	4	1540201	37442560	4.015783
case_076	5	1236760	38841548	5.001062
case_077	1	495320	0	1.000000
case_077	2	248734	57344	1.991364
case_077	3	168976	114688	2.931304
case_077	4	127832	114688	3.874773
case_077	5	115141	225046	4.301856
case_078	1	2296143	12644568	1.000000
case_078	2	1179848	11466240	1.946135
case_078	3	840459	12294144	2.732011
case_078	4	621588	14093136	3.693995
case_078	5	635493	14093136	3.613168
case_079	1	31052251	32448512	1.000000
case_079	2	14956594	242368512	2.076158
case_079	3	10106197	40280064	3.072595
case_079	4	7609995	31555584	4.080456
case_079	5	6022007	50976768	5.156462
case_080	1	292138	548864	1.000000
case_080	2	145757	2707456	2.004281
case_080	3	100325	3047424	2.911916
case_080	4	111314	4190208	2.624450
case_080	5	113455	4571136	2.574924
case_081	1	151795	0	1.000000
case_081	2	76860	0	1.974954
case_081	3	51943	0	2.922338
case_081	4	39588	0	3.834369
case_081	5	30943	0	4.905633
case_082	1	721041	0	1.000000
case_082	2	497483	5740350	1.449378
case_082	3	378014	7216752	1.907445
case_082	4	328283	5723710	2.196401
case_082	5	312290	5703230	2.308883
case_083	1	1057993	13934688	1.000000
case_083	2	543456	25772480	1.946787
case_083	3	436908	19450976	2.421546
case_083	4	417384	19407232	2.534819
case_083	5	391729	19753696	2.700829
case_084	1	2507412	0	1.000000
case_084	2	1254320	27648	1.999021
case_084	3	836116	18432	2.998881

算例	核数	Makespan	额外搬运量	加速比
case_084	4	629028	0	3.986169
case_084	5	503472	0	4.980241
case_085	1	3187899	0	1.000000
case_085	2	2334501	34301210	1.365559
case_085	3	1626396	34301210	1.960100
case_085	4	1324330	34301210	2.407179
case_085	5	1169044	34301210	2.726928
case_086	1	118403	0	1.000000
case_086	2	102016	2674522	1.160632
case_086	3	97297	3334272	1.216923
case_086	4	98592	3359618	1.200939
case_086	5	99507	3359618	1.189896
case_087	1	3974071	7902720	1.000000
case_087	2	1987606	10142208	1.999426
case_087	3	1374901	11036160	2.890442
case_087	4	1046846	12607488	3.796233
case_087	5	833349	12487680	4.768796
case_088	1	243623	0	1.000000
case_088	2	163881	2935364	1.486585
case_088	3	153688	2932484	1.585179
case_088	4	153194	2957060	1.590291
case_088	5	151301	2932484	1.610188
case_089	1	693092	0	1.000000
case_089	2	346832	238720	1.998351
case_089	3	243614	477440	2.845042
case_089	4	178160	716160	3.890278
case_089	5	147432	954880	4.701096
case_090	1	1178363	21233592	1.000000
case_090	2	625263	18405960	1.884588
case_090	3	550183	25354368	2.141766
case_090	4	527825	19402440	2.232488
case_090	5	509204	19405512	2.314128
case_091	1	9186769	53947008	1.000000
case_091	2	4571759	57904096	2.009460
case_091	3	3070918	60928832	2.991538
case_091	4	2315897	69360288	3.966830
case_091	5	1880163	73234432	4.886156
case_092	1	4815554	66000096	1.000000
case_092	2	2412929	113053056	1.995730
case_092	3	2109464	112651936	2.282833
case_092	4	2037129	113053056	2.363893
case_092	5	1661234	66980256	2.898781
case_093	1	74205	0	1.000000
case_093	2	37827	8192	1.961694
case_093	3	25103	16384	2.956021
case_093	4	19779	24576	3.751706
case_093	5	15876	32768	4.674036

算例	核数	Makespan	额外搬运量	加速比
case_094	1	94892	0	1.000000
case_094	2	47498	106272	1.997810
case_094	3	32728	212544	2.899413
case_094	4	24495	318552	3.873933
case_094	5	21804	424800	4.352045
case_095	1	2084853	0	1.000000
case_095	2	1042787	36864	1.999309
case_095	3	695775	0	2.996447
case_095	4	524422	0	3.975525
case_095	5	420852	0	4.953886
case_096	1	84194	0	1.000000
case_096	2	43782	116620	1.923028
case_096	3	34182	661880	2.463109
case_096	4	27087	719200	3.108281
case_096	5	27053	661824	3.112187
case_097	1	11491509	10584064	1.000000
case_097	2	5247114	85295104	2.190063
case_097	3	3509566	85327872	3.274339
case_097	4	2622226	14221312	4.382349
case_097	5	2101905	22016000	5.467188
case_098	1	177430	0	1.000000
case_098	2	93465	98340	1.898358
case_098	3	79351	1479456	2.236015
case_098	4	79805	1479456	2.223294
case_098	5	73104	3282976	2.427090
case_099	1	1622999	5484544	1.000000
case_099	2	808380	6221840	2.007718
case_099	3	540127	8245256	3.004847
case_099	4	405336	9838592	4.004083
case_099	5	334609	10674180	4.850434
case_100	1	151223	0	1.000000
case_100	2	80622	16896	1.875704
case_100	3	58758	698388	2.573658
case_100	4	58793	534528	2.572126
case_100	5	59649	534528	2.535214

C 问题二逐用例官方结果

每行对应一张图和一个核心数，共 500 行。

表 8 问题二逐例执行周期与额外搬运量

算例	核数	执行周期	额外搬运（字节）	加速比
1	1	233110	0	1.000000
1	2	116868	1152	1.994644
1	3	78545	2304	2.967853
1	4	58984	3456	3.952089
1	5	47502	4608	4.907372
2	1	261945	0	1.000000
2	2	134224	216576	1.951551
2	3	90565	423936	2.892343
2	4	68574	377856	3.819888
2	5	55189	514560	4.746326
3	1	602212	0	1.000000
3	2	341181	6564470	1.765081
3	3	274348	9497892	2.195066
3	4	217774	7333194	2.765307
3	5	199233	7908308	3.022652
4	1	392357	0	1.000000
4	2	199074	110592	1.970910
4	3	134043	221184	2.927098
4	4	104188	331776	3.765856
4	5	88341	442368	4.441392
5	1	95618	0	1.000000
5	2	61371	882310	1.558032
5	3	54267	1461452	1.761992
5	4	52462	1495922	1.822614
5	5	50673	1688564	1.886961
6	1	82837	16400	1.000000
6	2	40815	17280	2.029572
6	3	30449	118176	2.720516
6	4	26843	203696	3.085981
6	5	27216	219928	3.043688
7	1	1012562	0	1.000000
7	2	507494	27648	1.995220
7	3	338094	55296	2.994913
7	4	254398	82944	3.980228
7	5	203834	110592	4.967581
8	1	487605	0	1.000000
8	2	244266	0	1.996205
8	3	163359	0	2.984868
8	4	123060	0	3.962335
8	5	100603	0	4.846824
9	1	153261	0	1.000000
9	2	90801	547342	1.687878
9	3	72361	737384	2.118006

算例	核数	执行周期	额外搬运（字节）	加速比
9	4	62725	838982	2.443380
9	5	56088	898504	2.732510
10	1	86969	0	1.000000
10	2	42106	311944	2.065478
10	3	30856	48384	2.818544
10	4	26340	483232	3.301784
10	5	22006	562432	3.952059
11	1	184697	0	1.000000
11	2	92646	180224	1.993578
11	3	64180	1660928	2.877797
11	4	46880	540672	3.939782
11	5	46194	2033664	3.998290
12	1	59088	0	1.000000
12	2	25535	94960	2.314000
12	3	18128	189892	3.259488
12	4	14948	39168	3.952903
12	5	12519	25344	4.719866
13	1	263949	0	1.000000
13	2	132198	8192	1.996619
13	3	88351	16384	2.987504
13	4	66982	24576	3.940596
13	5	54363	32768	4.855306
14	1	17698626	71010720	1.000000
14	2	8909568	69998520	1.986474
14	3	5951263	56526504	2.973928
14	4	4526637	59878296	3.909884
14	5	3663108	62443416	4.831587
15	1	177362	180256	1.000000
15	2	87493	109824	2.027156
15	3	61018	298252	2.906716
15	4	48866	365824	3.629558
15	5	42569	326144	4.166459
16	1	7715523	0	1.000000
16	2	4027025	404200	1.915936
16	3	2877849	341100	2.681003
16	4	2244367	605676	3.437728
16	5	2088965	444880	3.693467
17	1	162810	22528	1.000000
17	2	76658	87552	2.123849
17	3	49850	175104	3.265998
17	4	39668	259200	4.104316
17	5	33081	331776	4.921556
18	1	1004367	1970176	1.000000
18	2	502491	2638848	1.998776
18	3	337417	3971072	2.976634
18	4	253676	4938752	3.959251
18	5	205917	5320704	4.877533

算例	核数	执行周期	额外搬运（字节）	加速比
19	1	81144	0	1.000000
19	2	38506	354360	2.107308
19	3	27716	54336	2.927695
19	4	22276	762456	3.642665
19	5	18267	255068	4.442109
20	1	532746	0	1.000000
20	2	266580	0	1.998447
20	3	177950	0	2.993796
20	4	133704	0	3.984518
20	5	107650	0	4.948871
21	1	6567937	4501504	1.000000
21	2	3174339	33017856	2.069072
21	3	2090922	34854912	3.141168
21	4	1550324	4608000	4.236493
21	5	1250168	11431936	5.253644
22	1	56780	0	1.000000
22	2	27860	145236	2.038047
22	3	18978	13824	2.991885
22	4	14336	20736	3.960658
22	5	11479	20736	4.946424
23	1	122614	0	1.000000
23	2	60965	74016	2.011220
23	3	40682	207264	3.013962
23	4	30516	281280	4.018023
23	5	24940	280992	4.916359
24	1	2555343	0	1.000000
24	2	1335245	396856	1.913763
24	3	954537	332124	2.677050
24	4	748435	595068	3.414248
24	5	695849	432232	3.672267
25	1	4863026	0	1.000000
25	2	2431840	27648	1.999731
25	3	1622164	55296	2.997863
25	4	1217444	82944	3.994456
25	5	975282	110592	4.986277
26	1	99203	0	1.000000
26	2	51429	21936	1.928931
26	3	41688	95664	2.379654
26	4	42226	172128	2.349335
26	5	38198	835212	2.597073
27	1	2363952	1400832	1.000000
27	2	1120962	1744896	2.108860
27	3	744579	18481152	3.174884
27	4	559196	3317760	4.227412
27	5	499622	21954560	4.731481
28	1	43836040	277412256	1.000000
28	2	20517451	75738432	2.136525

算例	核数	执行周期	额外搬运（字节）	加速比
28	3	13687852	44350128	3.202551
28	4	10347838	74883456	4.236251
28	5	8286707	73953792	5.289923
29	1	37555	0	1.000000
29	2	18880	16388	1.989142
29	3	12743	0	2.947108
29	4	10168	49164	3.693450
29	5	8787	65552	4.273927
30	1	2808034	5519360	1.000000
30	2	1390496	1292288	2.019448
30	3	933720	8985600	3.007362
30	4	697351	6002688	4.026715
30	5	559882	7484416	5.015403
31	1	1414369	3491328	1.000000
31	2	712262	5250048	1.985743
31	3	473822	7544832	2.985022
31	4	360134	7808000	3.927341
31	5	284848	7931904	4.965346
32	1	41443	0	1.000000
32	2	17566	6216	2.359274
32	3	11910	12360	3.479681
32	4	10728	74980	3.863069
32	5	10414	12360	3.979547
33	1	1774196	4592640	1.000000
33	2	889132	6452256	1.995425
33	3	596821	7053888	2.972744
33	4	443447	7214688	4.000920
33	5	361176	7894656	4.912275
34	1	842996	1079832	1.000000
34	2	415251	943616	2.030088
34	3	270947	2288640	3.111295
34	4	206454	3059200	4.083215
34	5	167389	3463168	5.036149
35	1	169571	0	1.000000
35	2	93692	801138	1.809877
35	3	75717	1276568	2.239537
35	4	63767	1505978	2.659228
35	5	58308	1058710	2.908194
36	1	931510	0	1.000000
36	2	466068	1152	1.998657
36	3	311345	2304	2.991890
36	4	233584	3456	3.987902
36	5	187182	4608	4.976493
37	1	210001	0	1.000000
37	2	105374	204800	1.992911
37	3	74038	573440	2.836395
37	4	53320	614400	3.938503

算例	核数	执行周期	额外搬运（字节）	加速比
37	5	43013	942080	4.882268
38	1	1364555	5627904	1.000000
38	2	674500	6790656	2.023062
38	3	449590	8107008	3.035110
38	4	338182	8954880	4.034972
38	5	275278	9592320	4.957007
39	1	1435262	7929856	1.000000
39	2	708430	9166848	2.025976
39	3	478013	11911168	3.002559
39	4	368961	14536704	3.890010
39	5	338925	14536704	4.234748
40	1	253454	0	1.000000
40	2	134642	1227916	1.882429
40	3	105492	1985768	2.402590
40	4	89539	1836692	2.830655
40	5	78938	1915296	3.210798
41	1	6003957	30898176	1.000000
41	2	2990398	33669120	2.007745
41	3	1991911	34851840	3.014169
41	4	1490603	39164928	4.027871
41	5	1196379	40630272	5.018441
42	1	1686740	6629376	1.000000
42	2	840111	7262496	2.007758
42	3	570685	8474176	2.955641
42	4	430510	8605536	3.918004
42	5	350713	9936736	4.809460
43	1	677656	49160	1.000000
43	2	349881	3210754	1.936819
43	3	250900	4852554	2.700901
43	4	207895	5894554	3.259607
43	5	199041	6609206	3.404605
44	1	154407	4438112	1.000000
44	2	54510	886880	2.832636
44	3	44484	891584	3.471068
44	4	49989	1389344	3.088820
44	5	43028	655488	3.588524
45	1	125614	0	1.000000
45	2	63059	0	1.992007
45	3	42601	0	2.948616
45	4	31878	0	3.940461
45	5	26164	0	4.801024
46	1	276455	3176288	1.000000
46	2	112410	942688	2.459345
46	3	96500	2958528	2.864819
46	4	71871	2098976	3.846545
46	5	77698	2660736	3.558071
47	1	326508	0	1.000000

算例	核数	执行周期	额外搬运（字节）	加速比
47	2	197413	1264006	1.653934
47	3	156690	1857324	2.083783
47	4	143256	2256146	2.279192
47	5	135565	2575320	2.408498
48	1	222220	0	1.000000
48	2	150049	467846	1.480983
48	3	114626	1177612	1.938653
48	4	114384	1270930	1.942754
48	5	101641	1273560	2.186322
49	1	196276	0	1.000000
49	2	126174	1027062	1.555598
49	3	83711	2408560	2.344686
49	4	75814	2607184	2.588915
49	5	76846	2907980	2.554147
50	1	149690	0	1.000000
50	2	87435	1022694	1.712015
50	3	60421	1831624	2.477450
50	4	57912	2053712	2.584784
50	5	57017	1337182	2.625357
51	1	607628	0	1.000000
51	2	319230	394084	1.903418
51	3	228581	328736	2.658261
51	4	183794	591064	3.306027
51	5	170016	427456	3.573946
52	1	180301	0	1.000000
52	2	90903	102912	1.983444
52	3	60160	181248	2.997025
52	4	45777	254976	3.938681
52	5	37846	333312	4.764070
53	1	1472686	5405536	1.000000
53	2	723016	4631434	2.036865
53	3	495435	6090084	2.972511
53	4	394500	8679750	3.733044
53	5	357068	9823040	4.124385
54	1	815051	0	1.000000
54	2	465795	2620302	1.749806
54	3	359793	4112418	2.265333
54	4	274779	7279264	2.966206
54	5	257061	8197794	3.170652
55	1	118835	69632	1.000000
55	2	52737	16384	2.253352
55	3	35943	74240	3.306207
55	4	31281	115712	3.798951
55	5	24831	173568	4.785752
56	1	277285	0	1.000000
56	2	172836	3326338	1.604324
56	3	122295	3278908	2.267345

算例	核数	执行周期	额外搬运（字节）	加速比
56	4	115731	3685886	2.395944
56	5	117134	4124830	2.367246
57	1	59726	94216	1.000000
57	2	29703	22304	2.010773
57	3	20565	14112	2.904255
57	4	16345	28224	3.654084
57	5	14028	14400	4.257628
58	1	4539717	28696576	1.000000
58	2	2266495	29548544	2.002968
58	3	1538252	30400512	2.951218
58	4	1141935	31551488	3.975460
58	5	1000800	32104448	4.536088
59	1	829473	1691648	1.000000
59	2	369318	360448	2.245959
59	3	254073	1171456	3.264703
59	4	185216	1081344	4.478409
59	5	150782	2072576	5.501141
60	1	888789	2068520	1.000000
60	2	441525	2746508	2.012998
60	3	295983	3159296	3.002838
60	4	223107	3960200	3.983689
60	5	180081	3733888	4.935496
61	1	77191	0	1.000000
61	2	39298	34560	1.964248
61	3	28342	43008	2.723555
61	4	23892	62208	3.230830
61	5	18840	110592	4.097187
62	1	2854809	0	1.000000
62	2	1431735	2347008	1.993951
62	3	958567	2268672	2.978205
62	4	722454	2603520	3.951544
62	5	579973	2746368	4.922314
63	1	1070397	0	1.000000
63	2	536070	1049088	1.996749
63	3	358763	1262592	2.983577
63	4	270138	1420800	3.962408
63	5	218917	1539072	4.889511
64	1	19116	0	1.000000
64	2	14130	47620	1.352866
64	3	13074	41446	1.462139
64	4	10160	130956	1.881496
64	5	11391	159314	1.678167
65	1	53068	0	1.000000
65	2	25517	13968	2.079712
65	3	17964	27936	2.954130
65	4	13800	41904	3.845507
65	5	11212	55872	4.733143

算例	核数	执行周期	额外搬运（字节）	加速比
66	1	359070	0	1.000000
66	2	203504	1653418	1.764437
66	3	146357	2500738	2.453385
66	4	116226	3806464	3.089412
66	5	106067	3979388	3.385313
67	1	63834834	264449408	1.000000
67	2	32040130	215994240	1.992340
67	3	21255269	159238912	3.003248
67	4	16409509	226600576	3.890112
67	5	13461153	229484544	4.742152
68	1	362095	0	1.000000
68	2	196346	1624576	1.844168
68	3	180284	2355384	2.008470
68	4	156990	2646854	2.306484
68	5	158423	5510878	2.285621
69	1	25144	0	1.000000
69	2	17030	364198	1.476453
69	3	14179	325098	1.773327
69	4	15620	361896	1.609731
69	5	13147	433008	1.912528
70	1	117300	0	1.000000
70	2	54493	129536	2.152570
70	3	39333	259072	2.982229
70	4	30080	333312	3.899601
70	5	25374	449024	4.622842
71	1	18919	0	1.000000
71	2	12313	171622	1.536506
71	3	9867	250696	1.917401
71	4	10732	223406	1.762859
71	5	10542	307214	1.794631
72	1	23897828	108613680	1.000000
72	2	11836023	113724192	2.019076
72	3	7941294	120851232	3.009312
72	4	5965868	123071264	4.005759
72	5	4826156	77245216	4.951731
73	1	10081626	141570432	1.000000
73	2	5294770	86179712	1.904073
73	3	3805931	134138624	2.648925
73	4	3295138	137012864	3.059546
73	5	2848898	94477312	3.538781
74	1	1682239	6070280	1.000000
74	2	814840	3776512	2.064502
74	3	542996	5619712	3.098069
74	4	408908	6623232	4.113979
74	5	328499	7151616	5.120987
75	1	1441809	0	1.000000
75	2	754663	4745798	1.910534

算例	核数	执行周期	额外搬运（字节）	加速比
75	3	557783	12507404	2.584892
75	4	462730	11096688	3.115875
75	5	378054	12427608	3.813765
76	1	6185113	28280832	1.000000
76	2	3065214	31095808	2.017841
76	3	2046322	17912832	3.022551
76	4	1527039	19563520	4.050396
76	5	1227673	22311936	5.038079
77	1	495320	0	1.000000
77	2	248734	57344	1.991364
77	3	168976	114688	2.931304
77	4	127832	114688	3.874773
77	5	114837	172032	4.313244
78	1	2296143	12644568	1.000000
78	2	1050081	2782080	2.186634
78	3	718909	6965760	3.193927
78	4	540759	6950784	4.246148
78	5	470359	10766304	4.881682
79	1	31052251	32448512	1.000000
79	2	15118882	194232320	2.053872
79	3	10181692	39960576	3.049812
79	4	7620379	31801344	4.074896
79	5	6063012	34084864	5.121588
80	1	292138	548864	1.000000
80	2	145825	1675264	2.003346
80	3	100325	3047424	2.911916
80	4	90584	2666496	3.225051
80	5	89345	3047424	3.269774
81	1	151795	0	1.000000
81	2	76860	0	1.974954
81	3	51943	0	2.922338
81	4	39588	0	3.834369
81	5	30943	0	4.905633
82	1	721041	0	1.000000
82	2	407535	3389668	1.769274
82	3	288553	5338828	2.498817
82	4	235778	6420240	3.058135
82	5	206503	7708408	3.491673
83	1	1057993	13934688	1.000000
83	2	442847	5589600	2.389071
83	3	290664	5739712	3.639918
83	4	220741	6471456	4.792916
83	5	201107	7741824	5.260846
84	1	2507412	0	1.000000
84	2	1175864	32004864	2.132400
84	3	836220	0	2.998508
84	4	629028	0	3.986169

算例	核数	执行周期	额外搬运（字节）	加速比
84	5	503472	0	4.980241
85	1	3187899	0	1.000000
85	2	1843091	17607566	1.729648
85	3	1276351	31680716	2.497666
85	4	995687	27902578	3.201708
85	5	890475	38398484	3.579998
86	1	118403	0	1.000000
86	2	74609	1019206	1.586980
86	3	63229	2197292	1.872606
86	4	62299	2353614	1.900560
86	5	60402	2265748	1.960250
87	1	3974071	7902720	1.000000
87	2	1987606	10142208	1.999426
87	3	1374901	11036160	2.890442
87	4	1046846	12607488	3.796233
87	5	833349	12487680	4.768796
88	1	243623	0	1.000000
88	2	132639	1032234	1.836737
88	3	122079	1572244	1.995618
88	4	112565	2028164	2.164287
88	5	112833	2034888	2.159147
89	1	693092	0	1.000000
89	2	329513	1467520	2.103383
89	3	225452	2429184	3.074233
89	4	165365	1827200	4.191286
89	5	133148	2186320	5.205426
90	1	1178363	21233592	1.000000
90	2	470442	3196848	2.504800
90	3	342250	5523456	3.442989
90	4	294798	7360608	3.997188
90	5	267434	7981056	4.406182
91	1	9186769	53947008	1.000000
91	2	4532609	52032556	2.026817
91	3	3042680	60113508	3.019302
91	4	2314443	67112896	3.969322
91	5	1872567	71035764	4.905976
92	1	4815554	66000096	1.000000
92	2	2017778	10740320	2.386563
92	3	1348585	21856448	3.570820
92	4	1018849	18053920	4.726465
92	5	826308	19945856	5.827795
93	1	74205	0	1.000000
93	2	37827	8192	1.961694
93	3	25103	16384	2.956021
93	4	19779	24576	3.751706
93	5	15876	32768	4.674036
94	1	94892	0	1.000000

算例	核数	执行周期	额外搬运（字节）	加速比
94	2	47498	106272	1.997810
94	3	32728	212544	2.899413
94	4	24668	318528	3.846765
94	5	21804	424800	4.352045
95	1	2084853	0	1.000000
95	2	1042890	0	1.999111
95	3	695775	0	2.996447
95	4	524422	0	3.975525
95	5	420852	0	4.953886
96	1	84194	0	1.000000
96	2	42443	346520	1.983696
96	3	30876	123264	2.726843
96	4	24217	234168	3.476649
96	5	22137	448896	3.803316
97	1	11491509	10584064	1.000000
97	2	5153667	2490368	2.229773
97	3	3444086	82444288	3.336592
97	4	2622394	14221312	4.382068
97	5	2102348	21311488	5.466036
98	1	177430	0	1.000000
98	2	89620	0	1.979804
98	3	65476	784240	2.709848
98	4	53735	961274	3.301945
98	5	47355	1166870	3.746806
99	1	1622999	5484544	1.000000
99	2	807983	5906432	2.008704
99	3	540063	6479872	3.005203
99	4	405336	9838592	4.004083
99	5	332266	7084052	4.884638
100	1	151223	0	1.000000
100	2	76850	621700	1.967768
100	3	57811	72192	2.615817
100	4	49623	1207894	3.047438
100	5	53383	1599352	2.832793

D 问题三逐用例成对结果

下列长表依次列出无 L2、固定方案 L2 与优化方案 L2 的执行周期，以及同配置缓存统计；三种周期均由题目评价程序给出，缓存统计由保存的事件账本复核。

表 9 问题三逐用例执行周期

用例	核数	无 L2	同方案 L2	优化 L2
1	1	233110	233110	233110
1	2	116868	116868	116868
1	3	78545	78545	78545
1	4	58984	58984	58984
1	5	47502	47502	47502
2	1	261945	261945	261945
2	2	134224	134224	134068
2	3	90565	90565	90565
2	4	68574	68574	68574
2	5	55189	55189	55189
3	1	602212	602212	602212
3	2	341181	340455	340046
3	3	274348	265160	265160
3	4	217774	210167	210167
3	5	199233	187997	187997
4	1	392357	392357	392357
4	2	199074	199074	199074
4	3	134043	134043	134043
4	4	104188	104188	104188
4	5	88341	88341	88341
5	1	95618	95618	95618
5	2	61371	60812	60812
5	3	54267	52965	52658
5	4	52462	49060	49060
5	5	50673	48259	48259
6	1	82837	82837	82837
6	2	40815	40815	40815
6	3	30449	30449	30449
6	4	26843	26843	26843
6	5	27216	27216	27216
7	1	1012562	1012562	1012562
7	2	507494	507494	507494
7	3	338094	338094	338094
7	4	254398	254398	254398
7	5	203834	203834	203834
8	1	487605	487605	487605
8	2	244266	244266	244266
8	3	163359	163359	163359
8	4	123060	123060	123060
8	5	100603	100603	100603
9	1	153261	153261	153261
9	2	90801	90800	90800

用例	核数	无 L2	同方案 L2	优化 L2
9	3	72361	72333	72333
9	4	62725	60949	60949
9	5	56088	53324	53324
10	1	86969	86969	86969
10	2	42106	42069	41842
10	3	30856	30856	30856
10	4	26340	25769	25583
10	5	22006	21371	21371
11	1	184697	184697	184697
11	2	92646	92646	92646
11	3	64180	64180	63580
11	4	46880	46880	46880
11	5	46194	43952	43594
12	1	59088	59088	59088
12	2	25535	25535	25535
12	3	18128	18128	17926
12	4	14948	14948	13663
12	5	12519	12460	12460
13	1	263949	263949	263949
13	2	132198	132198	132198
13	3	88351	88351	88351
13	4	66982	66982	66982
13	5	54363	54363	54363
14	1	17698626	17689602	17689602
14	2	8909568	8904096	8904096
14	3	5951263	5936435	5929070
14	4	4526637	4510849	4476730
14	5	3663108	3647377	3607077
15	1	177362	177362	177362
15	2	87493	87493	87493
15	3	61018	61018	61018
15	4	48866	48792	48792
15	5	42569	42569	42569
16	1	7715523	7715523	7715523
16	2	4027025	4026193	4026193
16	3	2877849	2877209	2877209
16	4	2244367	2240364	2240364
16	5	2088965	2086755	2086755
17	1	162810	162810	162810
17	2	76658	76501	76501
17	3	49850	49801	49801
17	4	39668	39668	39668
17	5	33081	33081	33081
18	1	1004367	1003110	1003110
18	2	502491	500447	500447
18	3	337417	335889	335889
18	4	253676	253384	253384

用例	核数	无 L2	同方案 L2	优化 L2
18	5	205917	203609	203609
19	1	81144	81144	81144
19	2	38506	38506	37982
19	3	27716	27716	27716
19	4	22276	21470	21470
19	5	18267	18126	18009
20	1	532746	532746	532746
20	2	266580	266580	266580
20	3	177950	177950	177950
20	4	133704	133704	133704
20	5	107650	107650	107650
21	1	6567937	6558837	6558837
21	2	3174339	3174060	3174060
21	3	2090922	2090922	2090922
21	4	1550324	1550324	1550324
21	5	1250168	1249380	1249380
22	1	56780	56780	56780
22	2	27860	27860	27860
22	3	18978	18978	18978
22	4	14336	14336	14336
22	5	11479	11479	11479
23	1	122614	122614	122614
23	2	60965	60965	60965
23	3	40682	40682	40682
23	4	30516	30516	30516
23	5	24940	24940	24940
24	1	2555343	2555343	2555343
24	2	1335245	1334413	1334413
24	3	954537	953897	953897
24	4	748435	744432	744432
24	5	695849	693639	693639
25	1	4863026	4863026	4863026
25	2	2431840	2431840	2431840
25	3	1622164	1622164	1622164
25	4	1217444	1217444	1217444
25	5	975282	975282	975282
26	1	99203	99203	99203
26	2	51429	51429	51429
26	3	41688	41688	41688
26	4	42226	42208	42208
26	5	38198	37332	36689
27	1	2363952	2359515	2359515
27	2	1120962	1120962	1120962
27	3	744579	744579	744579
27	4	559196	559196	559196
27	5	499622	494470	494412
28	1	43836040	43836040	43836040

用例	核数	无 L2	同方案 L2	优化 L2
28	2	20517451	20505375	20505375
28	3	13687852	13687122	13686058
28	4	10347838	10168619	10168619
28	5	8286707	8179833	8179833
29	1	37555	37555	37555
29	2	18880	18880	18880
29	3	12743	12743	12743
29	4	10168	10168	10168
29	5	8787	8787	8787
30	1	2808034	2807891	2807891
30	2	1390496	1390292	1390292
30	3	933720	933018	933018
30	4	697351	697321	697321
30	5	559882	559862	559862
31	1	1414369	1414159	1414159
31	2	712262	711649	711649
31	3	473822	473052	473052
31	4	360134	359705	359705
31	5	284848	284820	284820
32	1	41443	41443	41443
32	2	17566	17566	17495
32	3	11910	11910	11910
32	4	10728	10728	10728
32	5	10414	10384	10384
33	1	1774196	1774080	1774080
33	2	889132	887756	887756
33	3	596821	593725	593725
33	4	443447	442667	442667
33	5	361176	360597	360597
34	1	842996	842733	842733
34	2	415251	415212	415212
34	3	270947	270017	266836
34	4	206454	203915	202625
34	5	167389	165377	162156
35	1	169571	169571	169571
35	2	93692	93692	93692
35	3	75717	75628	75176
35	4	63767	63250	63250
35	5	58308	57848	57848
36	1	931510	931510	931510
36	2	466068	466068	466068
36	3	311345	311345	311345
36	4	233584	233584	233584
36	5	187182	187182	187182
37	1	210001	210001	210001
37	2	105374	105374	105374
37	3	74038	74038	74038

用例	核数	无 L2	同方案 L2	优化 L2
37	4	53320	53320	53320
37	5	43013	43013	43013
38	1	1364555	1363312	1363312
38	2	674500	674382	674382
38	3	449590	449048	449048
38	4	338182	337550	337550
38	5	275278	272573	272573
39	1	1435262	1435262	1435262
39	2	708430	706188	706188
39	3	478013	477811	477811
39	4	368961	367059	355737
39	5	338925	317103	317103
40	1	253454	253454	253454
40	2	134642	134149	134149
40	3	105492	105156	105156
40	4	89539	87558	87558
40	5	78938	77963	77963
41	1	6003957	6001575	6001575
41	2	2990398	2988290	2988290
41	3	1991911	1989953	1989953
41	4	1490603	1487676	1487676
41	5	1196379	1193054	1193054
42	1	1686740	1675633	1675633
42	2	840111	839453	839453
42	3	570685	567481	567481
42	4	430510	425563	425563
42	5	350713	348878	348878
43	1	677656	677656	677656
43	2	349881	349668	349668
43	3	250900	250731	250728
43	4	207895	204194	204135
43	5	199041	184463	182238
44	1	154407	107814	107814
44	2	54510	52355	51266
44	3	44484	42915	40954
44	4	49989	44212	41362
44	5	43028	39788	39677
45	1	125614	125614	125614
45	2	63059	63059	63059
45	3	42601	42601	42601
45	4	31878	31878	31878
45	5	26164	26164	26164
46	1	276455	244994	244994
46	2	112410	112410	112410
46	3	96500	93498	92335
46	4	71871	69926	66914
46	5	77698	71867	69923

用例	核数	无 L2	同方案 L2	优化 L2
47	1	326508	326508	326508
47	2	197413	197408	197408
47	3	156690	156593	156593
47	4	143256	142699	142699
47	5	135565	134070	134070
48	1	222220	222220	222220
48	2	150049	150049	150049
48	3	114626	114531	114531
48	4	114384	114072	114072
48	5	101641	101570	101570
49	1	196276	196276	196276
49	2	126174	126174	126174
49	3	83711	83597	83597
49	4	75814	75141	75141
49	5	76846	74748	74748
50	1	149690	149690	149690
50	2	87435	87422	87422
50	3	60421	60008	60008
50	4	57912	57018	57012
50	5	57017	56411	56411
51	1	607628	607628	607628
51	2	319230	318398	318398
51	3	228581	227941	227941
51	4	183794	179791	179791
51	5	170016	167806	167806
52	1	180301	180301	180301
52	2	90903	90903	90903
52	3	60160	60160	60160
52	4	45777	45777	45777
52	5	37846	37846	37846
53	1	1472686	1428109	1428109
53	2	723016	723023	723023
53	3	495435	495119	495119
53	4	394500	388744	388744
53	5	357068	343404	343404
54	1	815051	815051	815051
54	2	465795	465795	465795
54	3	359793	359773	359773
54	4	274779	273124	272727
54	5	257061	252980	252980
55	1	118835	118835	118835
55	2	52737	52632	52632
55	3	35943	35943	35943
55	4	31281	31281	31281
55	5	24831	24831	24831
56	1	277285	277285	277285
56	2	172836	172843	172799

用例	核数	无 L2	同方案 L2	优化 L2
56	3	122295	122170	122170
56	4	115731	114771	114771
56	5	117134	115146	115146
57	1	59726	59726	59726
57	2	29703	29703	29703
57	3	20565	20565	20565
57	4	16345	16345	16345
57	5	14028	14028	14028
58	1	4539717	4539717	4539717
58	2	2266495	2264017	2264017
58	3	1538252	1538118	1538118
58	4	1141935	1129512	1128131
58	5	1000800	960493	960493
59	1	829473	815938	815938
59	2	369318	369318	369318
59	3	254073	254073	253986
59	4	185216	185216	185216
59	5	150782	150736	150736
60	1	888789	887801	887801
60	2	441525	441112	441112
60	3	295983	295334	295334
60	4	223107	222447	222447
60	5	180081	179744	179744
61	1	77191	77191	77191
61	2	39298	39272	39272
61	3	28342	28316	28316
61	4	23892	23892	23892
61	5	18840	18834	18497
62	1	2854809	2854809	2854809
62	2	1431735	1431735	1431735
62	3	958567	958567	955896
62	4	722454	722454	722454
62	5	579973	579973	579973
63	1	1070397	1070397	1070397
63	2	536070	536070	536070
63	3	358763	358763	358763
63	4	270138	270138	270138
63	5	218917	218917	218917
64	1	19116	19116	19116
64	2	14130	14130	14130
64	3	13074	13074	13074
64	4	10160	10130	10130
64	5	11391	11356	11356
65	1	53068	53068	53068
65	2	25517	25411	25411
65	3	17964	17964	17964
65	4	13800	13800	13800

用例	核数	无 L2	同方案 L2	优化 L2
65	5	11212	11212	11212
66	1	359070	359070	359070
66	2	203504	203504	203504
66	3	146357	145116	145116
66	4	116226	112359	112359
66	5	106067	100249	100249
67	1	63834834	63834834	63834834
67	2	32040130	32217720	32217720
67	3	21255269	20814755	20814755
67	4	16409509	16150583	16150583
67	5	13461153	13133609	13133609
68	1	362095	362095	362095
68	2	196346	196325	196325
68	3	180284	179131	179131
68	4	156990	153012	153012
68	5	158423	143687	143687
69	1	25144	25144	25144
69	2	17030	16941	16940
69	3	14179	14019	13997
69	4	15620	15156	15156
69	5	13147	12656	12584
70	1	117300	117300	117300
70	2	54493	54493	53981
70	3	39333	39333	39333
70	4	30080	30080	30080
70	5	25374	25264	25264
71	1	18919	18919	18919
71	2	12313	12271	12271
71	3	9867	9692	9534
71	4	10732	10334	10214
71	5	10542	10249	10249
72	1	23897828	23890756	23890756
72	2	11836023	11826643	11826643
72	3	7941294	7929713	7929713
72	4	5965868	5953384	5953384
72	5	4826156	4803888	4803888
73	1	10081626	10081626	10081626
73	2	5294770	5154759	5154759
73	3	3805931	3497998	3497998
73	4	3295138	2862419	2862419
73	5	2848898	2661730	2578536
74	1	1682239	1678120	1678120
74	2	814840	813116	813116
74	3	542996	542102	542102
74	4	408908	408908	408908
74	5	328499	328279	328279
75	1	1441809	1441809	1441809

用例	核数	无 L2	同方案 L2	优化 L2
75	2	754663	754663	754663
75	3	557783	554461	554461
75	4	462730	457083	457083
75	5	378054	364732	364732
76	1	6185113	6183098	6183098
76	2	3065214	3063843	3063843
76	3	2046322	2045754	2045044
76	4	1527039	1526713	1526713
76	5	1227673	1226487	1226487
77	1	495320	495320	495320
77	2	248734	248734	248734
77	3	168976	168976	168976
77	4	127832	127591	127591
77	5	114837	114837	114837
78	1	2296143	2296143	2296143
78	2	1050081	1049840	1049840
78	3	718909	715241	711932
78	4	540759	532165	532165
78	5	470359	435870	435870
79	1	31052251	31052251	31052251
79	2	15118882	15118882	15116703
79	3	10181692	10175315	10175315
79	4	7620379	7612680	7612680
79	5	6063012	6062122	6062082
80	1	292138	291826	291826
80	2	145825	145636	145636
80	3	100325	100325	100325
80	4	90584	89925	89925
80	5	89345	85328	78076
81	1	151795	151795	151795
81	2	76860	76860	76860
81	3	51943	51943	51943
81	4	39588	39588	39588
81	5	30943	30943	30943
82	1	721041	721041	721041
82	2	407535	407535	407508
82	3	288553	287410	287410
82	4	235778	232569	232569
82	5	206503	200705	200705
83	1	1057993	946145	946145
83	2	442847	434269	434269
83	3	290664	290640	290640
83	4	220741	220741	220741
83	5	201107	194079	194079
84	1	2507412	2507412	2507412
84	2	1175864	1176757	1176657
84	3	836220	836220	836220

用例	核数	无 L2	同方案 L2	优化 L2
84	4	629028	629028	629028
84	5	503472	503472	503472
85	1	3187899	3187899	3187899
85	2	1843091	1842465	1842465
85	3	1276351	1264859	1264508
85	4	995687	989911	989911
85	5	890475	846640	846640
86	1	118403	118403	118403
86	2	74609	74274	74274
86	3	63229	61856	61671
86	4	62299	59945	59945
86	5	60402	57399	57355
87	1	3974071	3969779	3969779
87	2	1987606	1984150	1984150
87	3	1374901	1372451	1372451
87	4	1046846	1043492	1043492
87	5	833349	829669	829669
88	1	243623	243623	243623
88	2	132639	132093	132093
88	3	122079	119274	119274
88	4	112565	111170	109290
88	5	112833	109702	109702
89	1	693092	693092	693092
89	2	329513	329513	329513
89	3	225452	225450	225450
89	4	165365	165365	165365
89	5	133148	133139	133139
90	1	1178363	1178363	1178363
90	2	470442	462677	456438
90	3	342250	322129	309173
90	4	294798	259575	258592
90	5	267434	239128	227333
91	1	9186769	9168820	9168820
91	2	4532609	4531465	4531465
91	3	3042680	3029129	3029129
91	4	2314443	2289607	2289607
91	5	1872567	1837577	1837577
92	1	4815554	4333631	4333631
92	2	2017778	2013086	2013056
92	3	1348585	1349050	1345074
92	4	1018849	1013827	1005380
92	5	826308	807785	801052
93	1	74205	74205	74205
93	2	37827	37827	37827
93	3	25103	25103	25103
93	4	19779	19779	19779
93	5	15876	15876	15876

用例	核数	无 L2	同方案 L2	优化 L2
94	1	94892	94892	94892
94	2	47498	47498	47498
94	3	32728	32728	32728
94	4	24668	24668	24668
94	5	21804	21282	21282
95	1	2084853	2084853	2084853
95	2	1042890	1042890	1042890
95	3	695775	695775	695775
95	4	524422	524422	524422
95	5	420852	420852	420852
96	1	84194	84194	84194
96	2	42443	42426	42426
96	3	30876	30876	30876
96	4	24217	24217	24217
96	5	22137	21973	21973
97	1	11491509	11491509	11491509
97	2	5153667	5153650	5153650
97	3	3444086	3444086	3444086
97	4	2622394	2622413	2622413
97	5	2102348	2102250	2102250
98	1	177430	177430	177430
98	2	89620	89620	89620
98	3	65476	65195	65195
98	4	53735	53405	53405
98	5	47355	45200	45200
99	1	1622999	1618902	1618902
99	2	807983	806886	806886
99	3	540063	539170	539170
99	4	405336	403555	403555
99	5	332266	332266	332266
100	1	151223	151223	151223
100	2	76850	76561	76561
100	3	57811	57811	57811
100	4	49623	48347	48102
100	5	53383	48631	48041

同方案的逻辑额外搬运量与无 L2 相同；无 L2 命中率不适用。

表 10 问题三逐用例额外搬运字节与字节命中率

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
1	1	0	0	0.0000	0.0000
1	2	1152	1152	0.0000	0.0000
1	3	2304	2304	0.0000	0.0000
1	4	3456	3456	0.0000	0.0000
1	5	4608	4608	0.0000	0.0000
2	1	0	0	0.0000	0.0000

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
2	2	216576	245760	0.0372	0.0369
2	3	423936	423936	0.0000	0.0000
2	4	377856	377856	0.0000	0.0000
2	5	514560	514560	0.0031	0.0031
3	1	0	0	0.0000	0.0000
3	2	6564470	6564470	0.0743	0.0777
3	3	9497892	9497892	0.2174	0.2174
3	4	7333194	7333194	0.2996	0.2996
3	5	7908308	7908308	0.3395	0.3395
4	1	0	0	0.0000	0.0000
4	2	110592	110592	0.0000	0.0000
4	3	221184	221184	0.0000	0.0000
4	4	331776	331776	0.0000	0.0000
4	5	442368	442368	0.0000	0.0000
5	1	0	0	0.0000	0.0000
5	2	882310	882310	0.1366	0.1366
5	3	1461452	1465868	0.2530	0.2527
5	4	1495922	1495922	0.2631	0.2631
5	5	1688564	1688564	0.2921	0.2921
6	1	16400	16400	0.0293	0.0293
6	2	17280	17280	0.0308	0.0308
6	3	118176	118176	0.0837	0.0837
6	4	203696	203696	0.0979	0.0979
6	5	219928	219928	0.1174	0.1174
7	1	0	0	0.0000	0.0000
7	2	27648	27648	0.0000	0.0000
7	3	55296	55296	0.0000	0.0000
7	4	82944	82944	0.0000	0.0000
7	5	110592	110592	0.0000	0.0000
8	1	0	0	0.0000	0.0000
8	2	0	0	0.0000	0.0000
8	3	0	0	0.0000	0.0000
8	4	0	0	0.0000	0.0000
8	5	0	0	0.0000	0.0000
9	1	0	0	0.0000	0.0000
9	2	547342	547342	0.0663	0.0663
9	3	737384	737384	0.0851	0.0851
9	4	838982	838982	0.1867	0.1867
9	5	898504	898504	0.2349	0.2349
10	1	0	0	0.0000	0.0000
10	2	311944	300424	0.1418	0.1275
10	3	48384	48384	0.1119	0.1119
10	4	483232	485920	0.2698	0.2772
10	5	562432	562432	0.2929	0.2929
11	1	0	0	0.0000	0.0000
11	2	180224	180224	0.0000	0.0000
11	3	1660928	1660928	0.1861	0.1953

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
11	4	540672	540672	0.0000	0.0000
11	5	2033664	2033664	0.2430	0.2576
12	1	0	0	0.0000	0.0000
12	2	94960	94960	0.0204	0.0204
12	3	189892	189892	0.0673	0.0673
12	4	39168	39168	0.0396	0.0727
12	5	25344	25344	0.0497	0.0497
13	1	0	0	0.0000	0.0000
13	2	8192	8192	0.0000	0.0000
13	3	16384	16384	0.0000	0.0000
13	4	24576	24576	0.0000	0.0000
13	5	32768	32768	0.0000	0.0000
14	1	71010720	71010720	0.1229	0.1229
14	2	69998520	69998520	0.2013	0.2013
14	3	56526504	57041304	0.2471	0.2363
14	4	59878296	59878296	0.2981	0.2963
14	5	62443416	62443416	0.2880	0.3113
15	1	180256	180256	0.1693	0.1693
15	2	109824	109824	0.1105	0.1105
15	3	298252	298252	0.2522	0.2522
15	4	365824	365824	0.2140	0.2140
15	5	326144	326144	0.2694	0.2694
16	1	0	0	0.0000	0.0000
16	2	404200	404200	0.4965	0.4965
16	3	341100	341100	0.4512	0.4512
16	4	605676	605676	0.5964	0.5964
16	5	444880	444880	0.5163	0.5163
17	1	22528	22528	0.0404	0.0404
17	2	87552	87552	0.0617	0.0617
17	3	175104	175104	0.0735	0.0735
17	4	259200	259200	0.0464	0.0464
17	5	331776	331776	0.1183	0.1183
18	1	1970176	1970176	0.3438	0.3438
18	2	2638848	2638848	0.4835	0.4835
18	3	3971072	3971072	0.5357	0.5357
18	4	4938752	4938752	0.5624	0.5624
18	5	5320704	5320704	0.5712	0.5712
19	1	0	0	0.0000	0.0000
19	2	354360	370744	0.1209	0.1183
19	3	54336	54336	0.2326	0.2326
19	4	762456	762456	0.1934	0.1934
19	5	255068	279644	0.0784	0.3117
20	1	0	0	0.0000	0.0000
20	2	0	0	0.0000	0.0000
20	3	0	0	0.0000	0.0000
20	4	0	0	0.0000	0.0000
20	5	0	0	0.0000	0.0000

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
21	1	4501504	4501504	0.5006	0.5006
21	2	33017856	33017856	0.0150	0.0150
21	3	34854912	34854912	0.0165	0.0165
21	4	4608000	4608000	0.0000	0.0000
21	5	11431936	11431936	0.5700	0.5700
22	1	0	0	0.0000	0.0000
22	2	145236	145236	0.0749	0.0749
22	3	13824	13824	0.0608	0.0608
22	4	20736	20736	0.0885	0.0885
22	5	20736	20736	0.0885	0.0885
23	1	0	0	0.0000	0.0000
23	2	74016	74016	0.0006	0.0006
23	3	207264	207264	0.3232	0.3232
23	4	281280	281280	0.3932	0.3932
23	5	280992	280992	0.2555	0.2555
24	1	0	0	0.0000	0.0000
24	2	396856	396856	0.4988	0.4988
24	3	332124	332124	0.4534	0.4534
24	4	595068	595068	0.5988	0.5988
24	5	432232	432232	0.5188	0.5188
25	1	0	0	0.0000	0.0000
25	2	27648	27648	0.0000	0.0000
25	3	55296	55296	0.0000	0.0000
25	4	82944	82944	0.0000	0.0000
25	5	110592	110592	0.0000	0.0000
26	1	0	0	0.0000	0.0000
26	2	21936	21936	0.0429	0.0429
26	3	95664	95664	0.1637	0.1637
26	4	172128	172128	0.2232	0.2232
26	5	835212	835212	0.1869	0.2337
27	1	1400832	1400832	0.2689	0.2689
27	2	1744896	1744896	0.0890	0.0890
27	3	18481152	18481152	0.0599	0.0599
27	4	3317760	3317760	0.0000	0.0000
27	5	21954560	21948416	0.1943	0.1969
28	1	277412256	277412256	0.0000	0.0000
28	2	75738432	75738432	0.0383	0.0383
28	3	44350128	44353200	0.1475	0.1489
28	4	74883456	74883456	0.5881	0.5881
28	5	73953792	73953792	0.6104	0.6104
29	1	0	0	0.0000	0.0000
29	2	16388	16388	0.0000	0.0000
29	3	0	0	0.0000	0.0000
29	4	49164	49164	0.0000	0.0000
29	5	65552	65552	0.0000	0.0000
30	1	5519360	5519360	0.2599	0.2599
30	2	1292288	1292288	0.3451	0.3451

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
30	3	8985600	8985600	0.5365	0.5365
30	4	6002688	6002688	0.7014	0.7014
30	5	7484416	7484416	0.7449	0.7449
31	1	3491328	3491328	0.2606	0.2606
31	2	5250048	5250048	0.4305	0.4305
31	3	7544832	7544832	0.4998	0.4998
31	4	7808000	7808000	0.4349	0.4349
31	5	7931904	7931904	0.5328	0.5328
32	1	0	0	0.0000	0.0000
32	2	6216	6216	0.0218	0.0218
32	3	12360	12360	0.0424	0.0424
32	4	74980	74980	0.0385	0.0385
32	5	12360	12360	0.0424	0.0424
33	1	4592640	4592640	0.0963	0.0963
33	2	6452256	6452256	0.1705	0.1705
33	3	7053888	7053888	0.3033	0.3033
33	4	7214688	7214688	0.4267	0.4267
33	5	7894656	7894656	0.3110	0.3110
34	1	1079832	1079832	0.3090	0.3090
34	2	943616	943616	0.2927	0.2927
34	3	2288640	2288640	0.4821	0.4915
34	4	3059200	3083776	0.5684	0.5625
34	5	3463168	3487744	0.5879	0.5832
35	1	0	0	0.0000	0.0000
35	2	801138	801138	0.1547	0.1547
35	3	1276568	1254808	0.1948	0.1962
35	4	1505978	1505978	0.2872	0.2872
35	5	1058710	1058710	0.2730	0.2730
36	1	0	0	0.0000	0.0000
36	2	1152	1152	0.0000	0.0000
36	3	2304	2304	0.0000	0.0000
36	4	3456	3456	0.0000	0.0000
36	5	4608	4608	0.0000	0.0000
37	1	0	0	0.0000	0.0000
37	2	204800	204800	0.0000	0.0000
37	3	573440	573440	0.3537	0.3537
37	4	614400	614400	0.0000	0.0000
37	5	942080	942080	0.4222	0.4222
38	1	5627904	5627904	0.1736	0.1736
38	2	6790656	6790656	0.2788	0.2788
38	3	8107008	8107008	0.3442	0.3442
38	4	8954880	8954880	0.3324	0.3324
38	5	9592320	9592320	0.4060	0.4060
39	1	7929856	7929856	0.0000	0.0000
39	2	9166848	9166848	0.4136	0.4136
39	3	11911168	11911168	0.4837	0.4837
39	4	14536704	14848000	0.5318	0.5736

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
39	5	14536704	14536704	0.4645	0.4645
40	1	0	0	0.0000	0.0000
40	2	1227916	1227916	0.1273	0.1273
40	3	1985768	1985768	0.2179	0.2179
40	4	1836692	1836692	0.2695	0.2695
40	5	1915296	1915296	0.3204	0.3204
41	1	30898176	30898176	0.1016	0.1016
41	2	33669120	33669120	0.2225	0.2225
41	3	34851840	34851840	0.2605	0.2605
41	4	39164928	39164928	0.3438	0.3438
41	5	40630272	40630272	0.3177	0.3177
42	1	6629376	6629376	0.3187	0.3187
42	2	7262496	7262496	0.4130	0.4130
42	3	8474176	8474176	0.3819	0.3819
42	4	8605536	8605536	0.4043	0.4043
42	5	9936736	9936736	0.5569	0.5569
43	1	49160	49160	0.0332	0.0332
43	2	3210754	3210754	0.2306	0.2306
43	3	4852554	4852554	0.3110	0.3164
43	4	5894554	5894554	0.3500	0.3694
43	5	6609206	6592822	0.3748	0.3820
44	1	4438112	4438112	0.8200	0.8200
44	2	886880	885856	0.1451	0.1879
44	3	891584	1149632	0.2131	0.3724
44	4	1389344	1468192	0.2893	0.5055
44	5	655488	655488	0.2112	0.2112
45	1	0	0	0.0000	0.0000
45	2	0	0	0.0000	0.0000
45	3	0	0	0.0000	0.0000
45	4	0	0	0.0000	0.0000
45	5	0	0	0.0000	0.0000
46	1	3176288	3176288	0.7530	0.7530
46	2	942688	942688	0.0755	0.0755
46	3	2958528	2958528	0.1543	0.4851
46	4	2098976	2316064	0.1966	0.2580
46	5	2660736	2920832	0.2171	0.4623
47	1	0	0	0.0000	0.0000
47	2	1264006	1264006	0.1064	0.1064
47	3	1857324	1857324	0.2092	0.2092
47	4	2256146	2256146	0.2820	0.2820
47	5	2575320	2575320	0.3312	0.3312
48	1	0	0	0.0000	0.0000
48	2	467846	467846	0.1452	0.1452
48	3	1177612	1177612	0.1576	0.1576
48	4	1270930	1270930	0.1970	0.1970
48	5	1273560	1273560	0.1722	0.1722
49	1	0	0	0.0000	0.0000

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
49	2	1027062	1027062	0.0341	0.0341
49	3	2408560	2408560	0.0989	0.0989
49	4	2607184	2607184	0.1189	0.1189
49	5	2907980	2907980	0.1535	0.1535
50	1	0	0	0.0000	0.0000
50	2	1022694	1022694	0.0460	0.0460
50	3	1831624	1831624	0.1256	0.1256
50	4	2053712	2043408	0.1550	0.1536
50	5	1337182	1337182	0.1440	0.1440
51	1	0	0	0.0000	0.0000
51	2	394084	394084	0.4997	0.4997
51	3	328736	328736	0.4543	0.4543
51	4	591064	591064	0.5997	0.5997
51	5	427456	427456	0.5197	0.5197
52	1	0	0	0.0000	0.0000
52	2	102912	102912	0.1127	0.1127
52	3	181248	181248	0.1828	0.1828
52	4	254976	254976	0.2264	0.2264
52	5	333312	333312	0.2369	0.2369
53	1	5405536	5405536	0.6249	0.6249
53	2	4631434	4631434	0.2397	0.2397
53	3	6090084	6090084	0.2527	0.2527
53	4	8679750	8679750	0.2221	0.2221
53	5	9823040	9823040	0.2537	0.2537
54	1	0	0	0.0000	0.0000
54	2	2620302	2620302	0.0559	0.0559
54	3	4112418	4112418	0.1204	0.1204
54	4	7279264	7284480	0.1450	0.1450
54	5	8197794	8197794	0.1659	0.1659
55	1	69632	69632	0.1001	0.1001
55	2	16384	16384	0.0255	0.0255
55	3	74240	74240	0.1061	0.1061
55	4	115712	115712	0.1188	0.1188
55	5	173568	173568	0.1653	0.1653
56	1	0	0	0.0000	0.0000
56	2	3326338	3326338	0.0445	0.0451
56	3	3278908	3278908	0.0939	0.0939
56	4	3685886	3685886	0.1385	0.1385
56	5	4124830	4124830	0.1673	0.1673
57	1	94216	94216	0.2187	0.2187
57	2	22304	22304	0.0621	0.0621
57	3	14112	14112	0.0008	0.0008
57	4	28224	28224	0.0774	0.0774
57	5	14400	14400	0.0016	0.0016
58	1	28696576	28696576	0.0000	0.0000
58	2	29548544	29548544	0.1157	0.1157
58	3	30400512	30400512	0.0538	0.0538

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
58	4	31551488	31821824	0.4236	0.4247
58	5	32104448	32104448	0.3095	0.3095
59	1	1691648	1691648	0.6100	0.6100
59	2	360448	360448	0.0000	0.0000
59	3	1171456	1171456	0.4000	0.5191
59	4	1081344	1081344	0.0000	0.0000
59	5	2072576	2072576	0.5442	0.5442
60	1	2068520	2068520	0.4231	0.4231
60	2	2746508	2746508	0.4789	0.4789
60	3	3159296	3159296	0.4754	0.4754
60	4	3960200	3960200	0.4592	0.4592
60	5	3733888	3733888	0.5229	0.5229
61	1	0	0	0.0000	0.0000
61	2	34560	34560	0.0597	0.0597
61	3	43008	43008	0.0733	0.0733
61	4	62208	62208	0.0709	0.0709
61	5	110592	110592	0.1583	0.1583
62	1	0	0	0.0000	0.0000
62	2	2347008	2325504	0.0000	0.0009
62	3	2268672	2268672	0.0000	0.0000
62	4	2603520	2582016	0.0012	0.0012
62	5	2746368	2746368	0.0000	0.0000
63	1	0	0	0.0000	0.0000
63	2	1049088	1049088	0.0057	0.0057
63	3	1262592	1262592	0.0063	0.0063
63	4	1420800	1420800	0.0086	0.0086
63	5	1539072	1539072	0.0100	0.0100
64	1	0	0	0.0000	0.0000
64	2	47620	47620	0.0256	0.0256
64	3	41446	41446	0.1366	0.1366
64	4	130956	130956	0.2073	0.2073
64	5	159314	159314	0.2249	0.2249
65	1	0	0	0.0000	0.0000
65	2	13968	13968	0.0445	0.0445
65	3	27936	27936	0.0431	0.0431
65	4	41904	41904	0.0822	0.0822
65	5	55872	55872	0.1312	0.1312
66	1	0	0	0.0000	0.0000
66	2	1653418	1653418	0.0916	0.0916
66	3	2500738	2500738	0.2189	0.2189
66	4	3806464	3806464	0.2231	0.2231
66	5	3979388	3979388	0.2264	0.2264
67	1	264449408	264449408	0.0000	0.0000
67	2	215994240	215994240	0.2151	0.2151
67	3	159238912	159238912	0.6616	0.6616
67	4	226600576	226600576	0.3433	0.3433
67	5	229484544	229484544	0.3472	0.3472

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
68	1	0	0	0.0000	0.0000
68	2	1624576	1624576	0.1013	0.1013
68	3	2355384	2355384	0.1847	0.1847
68	4	2646854	2646854	0.0877	0.0877
68	5	5510878	5510878	0.1944	0.1944
69	1	0	0	0.0000	0.0000
69	2	364198	364006	0.1287	0.1291
69	3	325098	325098	0.2892	0.2964
69	4	361896	361896	0.2733	0.2733
69	5	433008	437616	0.2238	0.2516
70	1	0	0	0.0000	0.0000
70	2	129536	129536	0.0513	0.1463
70	3	259072	259072	0.2730	0.2730
70	4	333312	333312	0.3419	0.3419
70	5	449024	449024	0.3041	0.3041
71	1	0	0	0.0000	0.0000
71	2	171622	171622	0.1003	0.1003
71	3	250696	246088	0.3087	0.3114
71	4	223406	223406	0.1782	0.2041
71	5	307214	307214	0.2663	0.2663
72	1	108613680	108613680	0.0262	0.0262
72	2	113724192	113724192	0.1242	0.1242
72	3	120851232	120851232	0.1151	0.1151
72	4	123071264	123071264	0.1695	0.1695
72	5	77245216	77245216	0.2340	0.2340
73	1	141570432	141570432	0.0000	0.0000
73	2	86179712	86179712	0.2207	0.2207
73	3	134138624	134138624	0.3086	0.3086
73	4	137012864	137012864	0.3213	0.3213
73	5	94477312	96247040	0.2336	0.3236
74	1	6070280	6070280	0.1668	0.1668
74	2	3776512	3776512	0.3465	0.3465
74	3	5619712	5619712	0.4595	0.4595
74	4	6623232	6623232	0.4171	0.4171
74	5	7151616	7151616	0.4715	0.4715
75	1	0	0	0.0000	0.0000
75	2	4745798	4745798	0.1045	0.1045
75	3	12507404	12507404	0.2335	0.2335
75	4	11096688	11096688	0.2466	0.2466
75	5	12427608	12427608	0.2615	0.2615
76	1	28280832	28280832	0.1336	0.1336
76	2	31095808	31095808	0.2788	0.2788
76	3	17912832	17794048	0.5245	0.5505
76	4	19563520	19563520	0.5526	0.5526
76	5	22311936	22311936	0.6242	0.6242
77	1	0	0	0.0000	0.0000
77	2	57344	57344	0.0261	0.0261

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
77	3	114688	114688	0.0254	0.0254
77	4	114688	114688	0.0508	0.0508
77	5	172032	172032	0.0744	0.0744
78	1	12644568	12644568	0.0000	0.0000
78	2	2782080	2782080	0.3321	0.3321
78	3	6965760	6965760	0.3449	0.4940
78	4	6950784	6950784	0.6988	0.6988
78	5	10766304	10766304	0.6982	0.6982
79	1	32448512	32448512	0.0000	0.0000
79	2	194232320	193798144	0.0009	0.0011
79	3	39960576	39960576	0.1373	0.1373
79	4	31801344	31801344	0.2438	0.2438
79	5	34084864	34283520	0.4735	0.4624
80	1	548864	548864	0.1686	0.1686
80	2	1675264	1675264	0.3902	0.3902
80	3	3047424	3047424	0.0000	0.0000
80	4	2666496	2666496	0.4597	0.4597
80	5	3047424	3047424	0.2051	0.4359
81	1	0	0	0.0000	0.0000
81	2	0	0	0.0000	0.0000
81	3	0	0	0.0000	0.0000
81	4	0	0	0.0000	0.0000
81	5	0	0	0.0000	0.0000
82	1	0	0	0.0000	0.0000
82	2	3389668	3389668	0.0594	0.0733
82	3	5338828	5338828	0.1725	0.1725
82	4	6420240	6420240	0.2422	0.2422
82	5	7708408	7708408	0.3140	0.3140
83	1	13934688	13934688	0.7959	0.7959
83	2	5589600	5589600	0.1595	0.1595
83	3	5739712	5739712	0.1416	0.1416
83	4	6471456	6471456	0.3448	0.3448
83	5	7741824	7741824	0.1860	0.1860
84	1	0	0	0.0000	0.0000
84	2	32004864	32000640	0.0052	0.0052
84	3	0	0	0.0000	0.0000
84	4	0	0	0.0000	0.0000
84	5	0	0	0.0000	0.0000
85	1	0	0	0.0000	0.0000
85	2	17607566	17607566	0.0753	0.0753
85	3	31680716	31680716	0.1996	0.2009
85	4	27902578	27902578	0.1863	0.1863
85	5	38398484	38398484	0.2762	0.2762
86	1	0	0	0.0000	0.0000
86	2	1019206	1019206	0.1717	0.1717
86	3	2197292	2183372	0.1747	0.1816
86	4	2353614	2353614	0.2334	0.2334

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
86	5	2265748	2265748	0.2584	0.2632
87	1	7902720	7902720	0.1834	0.1834
87	2	10142208	10142208	0.2955	0.2955
87	3	11036160	11036160	0.2615	0.2615
87	4	12607488	12607488	0.2995	0.2995
87	5	12487680	12487680	0.3022	0.3022
88	1	0	0	0.0000	0.0000
88	2	1032234	1032234	0.0924	0.0924
88	3	1572244	1572244	0.2117	0.2117
88	4	2028164	1954436	0.0682	0.0808
88	5	2034888	2034888	0.0803	0.0803
89	1	0	0	0.0000	0.0000
89	2	1467520	1467520	0.0194	0.0194
89	3	2429184	2429184	0.1780	0.1780
89	4	1827200	1827200	0.1027	0.1027
89	5	2186320	2186320	0.1429	0.1429
90	1	21233592	21233592	0.0000	0.0000
90	2	3196848	3196848	0.2160	0.2933
90	3	5523456	5524224	0.2970	0.4317
90	4	7360608	7550112	0.3184	0.3847
90	5	7981056	8312448	0.3274	0.4516
91	1	53947008	53947008	0.1054	0.1054
91	2	52032556	52032556	0.3523	0.3523
91	3	60113508	60113508	0.4441	0.4441
91	4	67112896	67112896	0.4855	0.4855
91	5	71035764	71035764	0.5336	0.5336
92	1	66000096	66000096	0.7935	0.7935
92	2	10740320	10740320	0.1172	0.1173
92	3	21856448	21848256	0.1120	0.1331
92	4	18053920	18267936	0.1479	0.2199
92	5	19945856	20375936	0.2219	0.2708
93	1	0	0	0.0000	0.0000
93	2	8192	8192	0.0000	0.0000
93	3	16384	16384	0.0000	0.0000
93	4	24576	24576	0.0000	0.0000
93	5	32768	32768	0.0000	0.0000
94	1	0	0	0.0000	0.0000
94	2	106272	106272	0.0563	0.0563
94	3	212544	212544	0.3108	0.3108
94	4	318528	318528	0.3624	0.3624
94	5	424800	424800	0.2478	0.2478
95	1	0	0	0.0000	0.0000
95	2	0	0	0.0000	0.0000
95	3	0	0	0.0000	0.0000
95	4	0	0	0.0000	0.0000
95	5	0	0	0.0000	0.0000
96	1	0	0	0.0000	0.0000

用例	核数	基线额外字节	优化额外字节	同方案命中率	优化命中率
96	2	346520	346520	0.2773	0.2773
96	3	123264	123264	0.2038	0.2038
96	4	234168	234168	0.1969	0.1969
96	5	448896	448896	0.3090	0.3090
97	1	10584064	10584064	0.0000	0.0000
97	2	2490368	2490368	0.2629	0.2629
97	3	82444288	82444288	0.0535	0.0535
97	4	14221312	14221312	0.0903	0.0903
97	5	21311488	21311488	0.2452	0.2452
98	1	0	0	0.0000	0.0000
98	2	0	0	0.0000	0.0000
98	3	784240	784240	0.2601	0.2601
98	4	961274	961274	0.3211	0.3211
98	5	1166870	1166870	0.3037	0.3037
99	1	5484544	5484544	0.2524	0.2524
99	2	5906432	5906432	0.4562	0.4562
99	3	6479872	6479872	0.4862	0.4862
99	4	9838592	9838592	0.5568	0.5568
99	5	7084052	7084052	0.6682	0.6682
100	1	0	0	0.0000	0.0000
100	2	621700	621700	0.2955	0.2955
100	3	72192	72192	0.0171	0.0171
100	4	1207894	1228630	0.4199	0.4249
100	5	1599352	1580920	0.4154	0.4202